INFO 490 MG:
A. I.
Web-Application Development

MOHIT GUPTA

Week 4:
Advanced Charts +
External APIs +
Django Authentication +
Google OAuth Part-1

Index

1. Charts built on Internal APIs
2. External APIs
3. Export (CSV / JSON)
4. Vega-Lite (Advanced Charts)
5. User Authentication (Django auth, sessions)
6. Code Cleanup

How many of you will be working with these features?

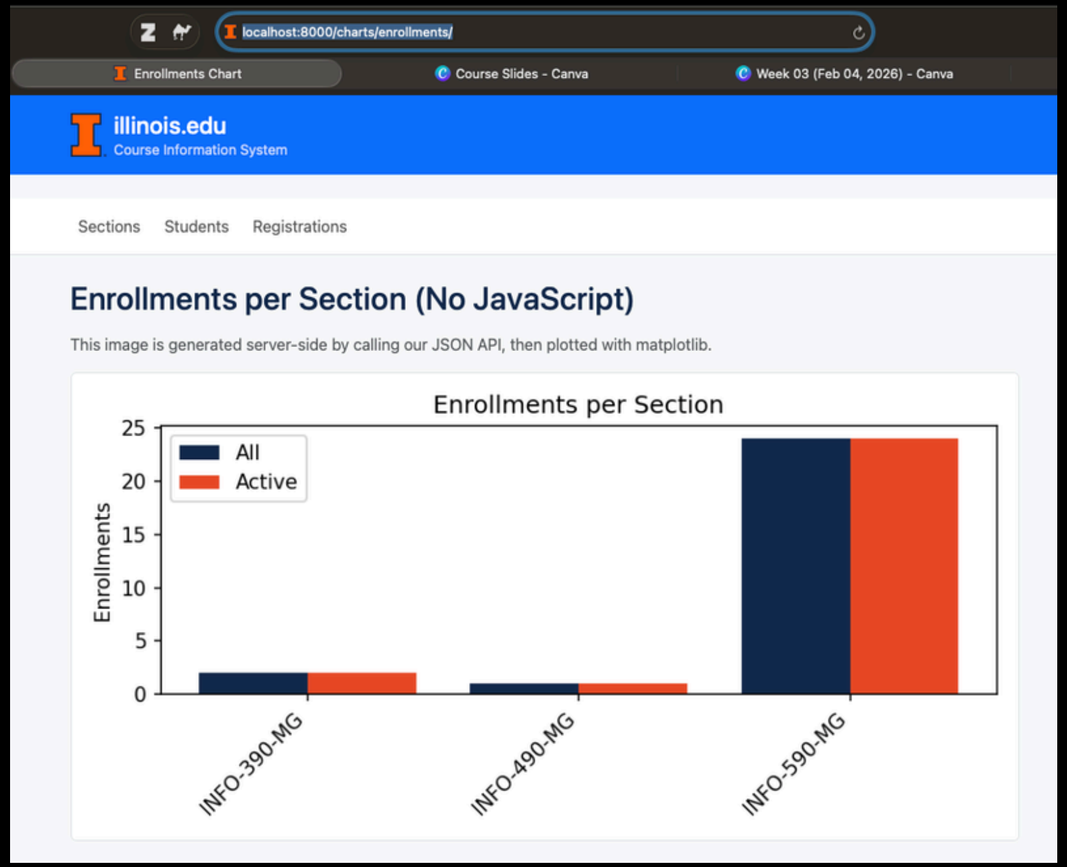
All the teams, how is your app development progress so far?

Progress Check!

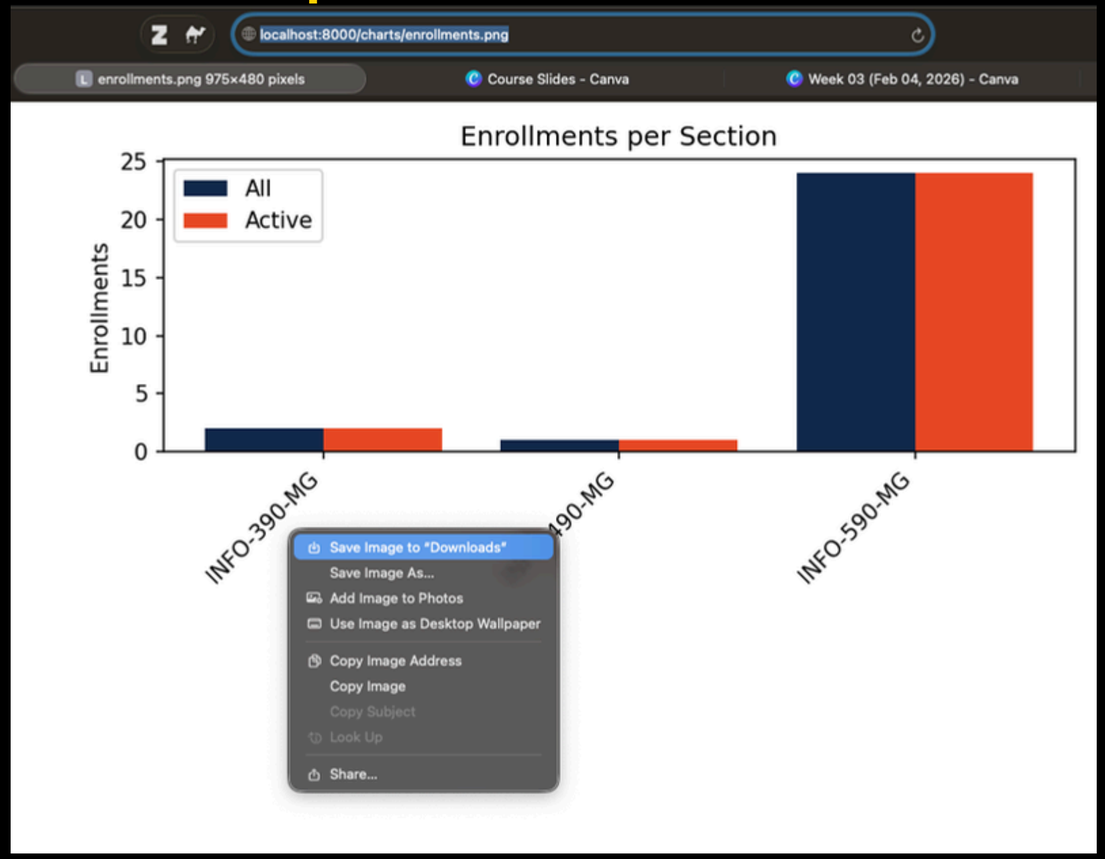
Activity time!

1. Charts built on Internal APIs

Our output-1 by the end of section 1.



And our output-2.

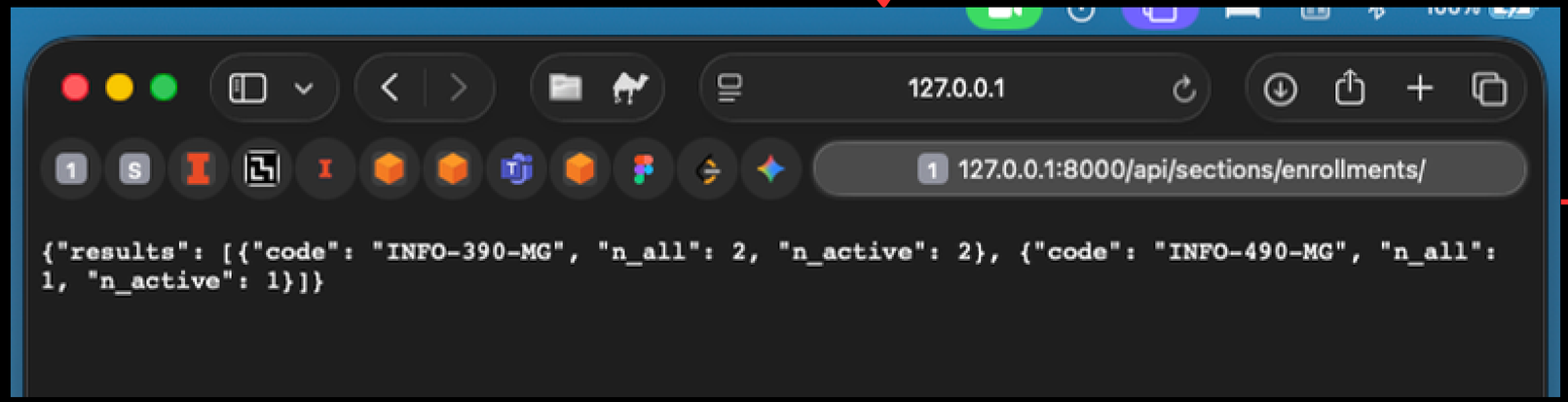


Let’s begin development part of this section.

This is the data that we calculated for enrollments per section

We are now going to indirectly access this data to create our charts

```
35
    /api/sections/enrollments/
36  path( route: "api/sections/enrollments/", views.api_enrollments_per_section, name="api-enrollments-per-section"),
37
38
```




```
{
  "results": [
    {
      "code": "INFO-390-MG",
      "n_all": 2,
      "n_active": 2
    },
    {
      "code": "INFO-490-MG",
      "n_all": 1,
      "n_active": 1
    }
  ]
}
```

Expected data shape that we are pulling from API

```
394  def enrollments_chart_png(request):
395      """
396      Server-side fetch of our own JSON API (no JS in the browser),
397      then build a matplotlib PNG and return it.
398      Expected API shape (from our earlier example):
399        {
400          "rows": [
401            {"code": "INFO-390-MG", "n_enrolls": 2, "n_active": 2},
402            {"code": "INFO-490-MG", "n_enrolls": 1, "n_active": 1}
403          ]
404        }
405      """
```

“CREATING” the URL link from which we’ll pull the data

```
406  # --- 1) DATA: fetch rows from your own JSON API ----------------------------
407  # Build absolute URL to your API (works in dev too)
408  # The below code is just work likes this:
409
410  # We do not write our local machine's url as it is static and not dynamic
411  # And we do not want to bother ourselves for changing the url again-and-again
412  # api_url = request.build_absolute_uri(reverse("http://127.0.0.1:8000/api/sections/enrollments/"))
413  # api_url = request.build_absolute_uri(reverse("http://www.illinois.edu/api/sections/enrollments/"))
414
415  api_url = request.build_absolute_uri(reverse("api-enrollments-per-section"))
416  # Output:
417  # api_url = http://127.0.0.1:8000/api/sections/enrollments/
418  # or
419  # api_url = http://www.illinois.edu/api/sections/enrollments/
420
421  #   •   reverse():
422  #       •   It is equivalent to using {% url 'api-enrollments-per-section' %} in a template.
423
424  #   •   request.build_absolute_uri():
425  #       •   This method takes a relative path (like /api/sections/enrollments/)
426  #           and builds a full absolute URL including the current domain.
427
```

Why uri?

“CREATING” the URL link from which we’ll pull the data

```
406  # --- 1) DATA: fetch rows from your own JSON API ----------------------------
407  # Build absolute URL to your API (works in dev too)
408  # The below code is just work likes this:
409
410  # We do not write our local machine's url as it is static and not dynamic
411  # And we do not want to bother ourselves for changing the url again-and-again
412  # api_url = request.build_absolute_uri(reverse("http://127.0.0.1:8000/api/sections/enrollments/"))
413  # api_url = request.build_absolute_uri(reverse("http://www.illinois.edu/api/sections/enrollments/"))
414
415  api_url = request.build_absolute_uri(reverse("api-enrollments-per-section"))
416  # Output:
417  # api_url = http://127.0.0.1:8000/api/sections/enrollments/
418  # or
419  # api_url = http://www.illinois.edu/api/sections/enrollments/
420
421  #   •   reverse():
422  #       •   It is equivalent to using {% url 'api-enrollments-per-section' %} in a template.
423
424  #   •   request.build_absolute_uri():
425  #       •   This method takes a relative path (like /api/sections/enrollments/)
426  #           and builds a full absolute URL including the current domain.
427
```

Why uri:
Because we are creating the name, but not fetching it

URL

Uniform Resource Locator
Eg:
http://127.0.0.1:8000/api/sections/enrollments/

- https://example.com/data.json → URL (you can fetch it)

- Every URL is a URI

VS

URI

- Uniform Resource Identifier
- Eg:
  - http://127.0.0.1:8000/api/sections/enrollments
  - urn:isbn:9780131103627 → URI, but not a URL (it identifies a book, not a web address).

- Every URI is not a URL
- It's not necessary that you can fetch it

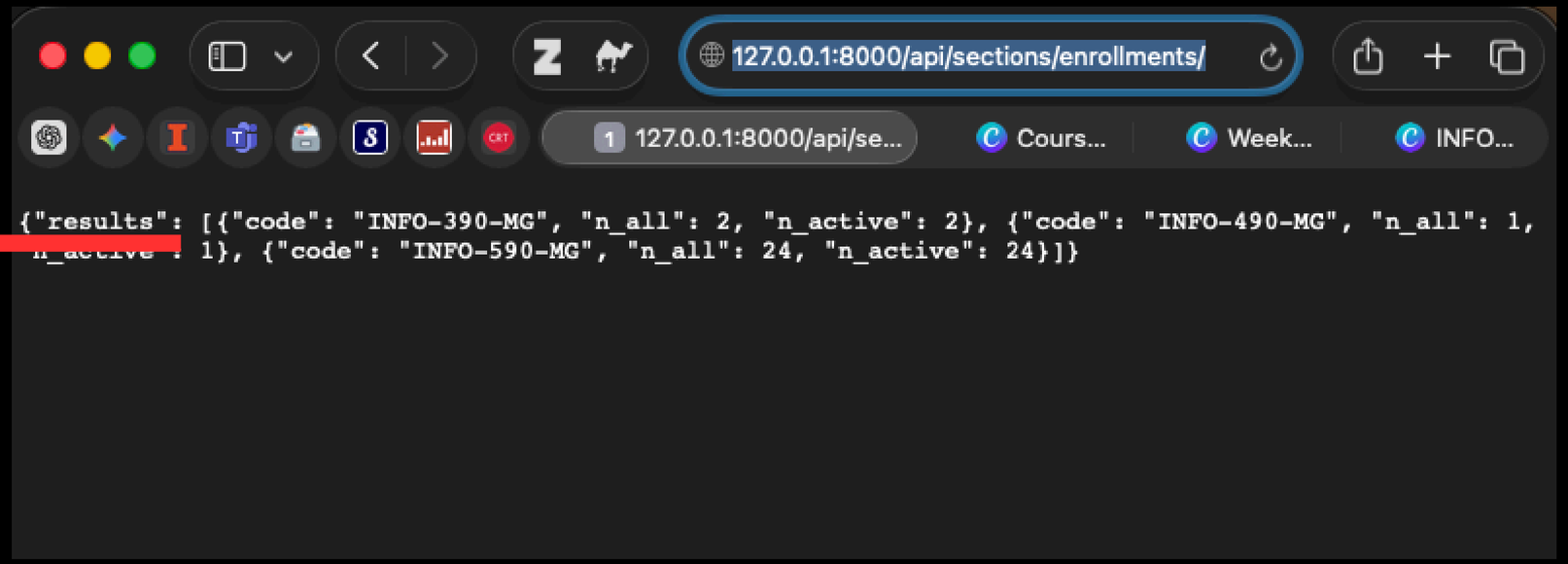
Analogy	URI	URL
Person	Their name	their home address
Book	Its ISBN	its library shelf location
Django object	Its primary key	its detail page URL

```
URI  (Uniform Resource Identifier)
 │
 ├── URL  (Locator) → tells WHERE
 │     e.g., https://example.com/page
 │
 └── URN  (Name)    → tells WHAT
       e.g., urn:isbn:9780131103627
```

Now instead of creating name through uri,
we are FETCHING the data from URL.

```
428     # Pull JSON from the API
429     with urllib.request.urlopen(api_url) as resp:
430         payload = json.load(resp)
431
```

The name of my table is results here.



Our current API output format.

Our desired format

```
# --- 2) PREPARING DATA: save columns into separate lists -----------------

#   •   payload is the Python dictionary you got from json.load(resp).
#   •   .get("rows", []) safely extracts the value associated with "rows".
# If "rows" is missing for any reason (bad response, empty API, etc.), it defaults to an empty list ([]).

# Right now, our data looks like a dictionary having "rows" as a single "key"
# and all the columns as dictionaries in a list. For eg: { "a": [ {}, {}, {} ] }
#
##   {
##   "results": [
##     {"code": "INFO-390-MG", "n_enrolls": 2, "n_active": 2},
##     {"code": "INFO-490-MG", "n_enrolls": 1, "n_active": 1}
##   ]
##   }

# We want to extract it such that we are just left with the nested list
##   "results": [
##             {"code": "INFO-390-MG", "n_enrolls": 2, "n_active": 2},
##             {"code": "INFO-490-MG", "n_enrolls": 1, "n_active": 1}
##          ]

# IMPORTANT:
# If you ever get a blank visualization but everything works, it might be because of no data being given to your charts
# It might be because the name of your table might be different.
# For me, here it is "results"
rows = payload.get("results", [])
```

now saving the data in separate variables

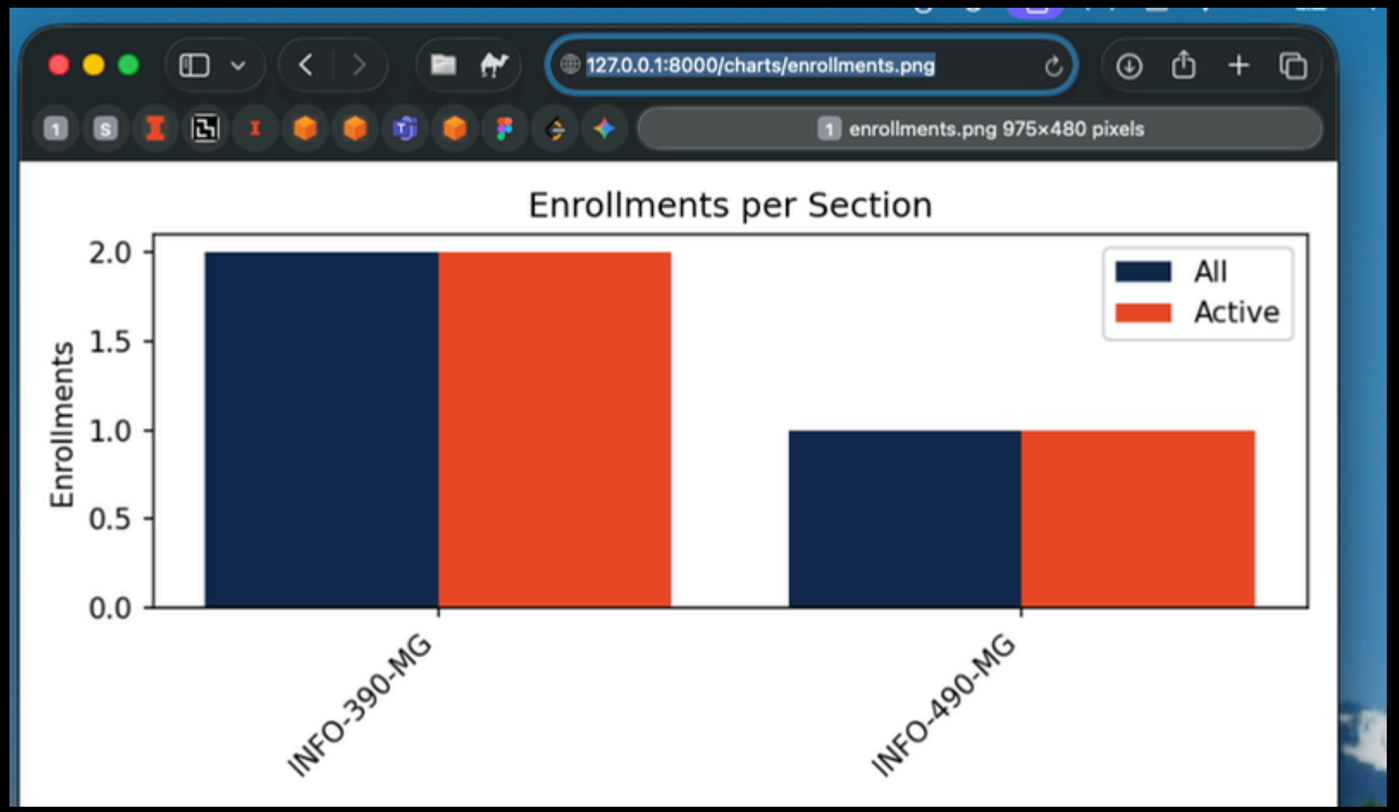
```
460    # This is a list comprehension, a compact way to loop through all rows and extract each section's code.
461    # labels = ["INFO-390-MG", "INFO-490-MG"]
462    labels       = [r["code"] for r in rows]
463    all_counts   = [r["n_all"] for r in rows]
464    active_counts= [r["n_active"]  for r in rows]
465
```

Creating the presentation

```
466  # --- 3) PRESENTATION: turn rows into a PNG with matplotlib ------------------
467  # Plot: grouped bars (All vs Active)
468  x = range(len(labels))
469  width = 0.4
470
471  fig, ax = plt.subplots(figsize=(6.5, 3.2), dpi=150)
472  ax.bar([i - width/2 for i in x], all_counts,    width=width, label="All",    color="#13294B")
473  ax.bar([i + width/2 for i in x], active_counts, width=width, label="Active", color="#E84A27")
474
475  ax.set_title("Enrollments per Section")
476  ax.set_ylabel("Enrollments")
477  ax.set_xticks(list(x))
478  ax.set_xticklabels(labels, rotation=45, ha="right")
479  ax.legend()
480  fig.tight_layout()
481
482  buf = BytesIO()
483  fig.savefig(buf, format="png")
484  plt.close(fig)
485  buf.seek(0)
486  return HttpResponse(buf.getvalue(), content_type="image/png")
487
488
```

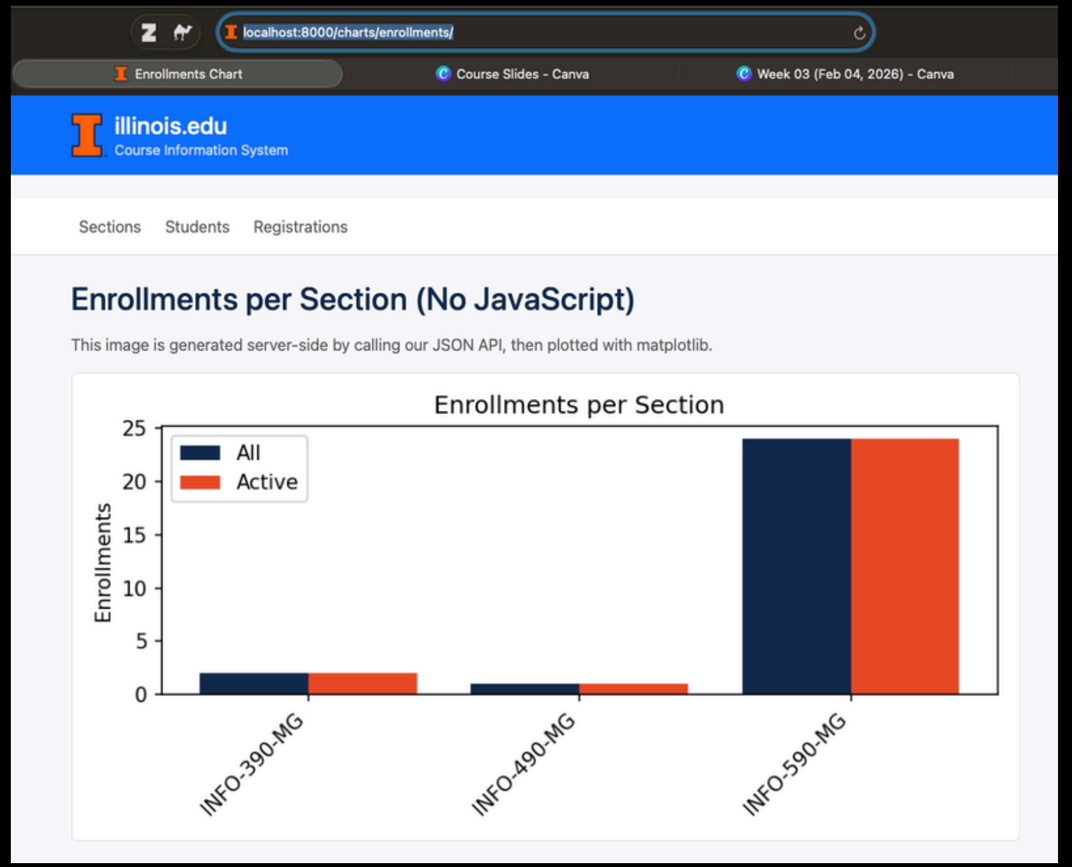
You can directly project a chart (like we did earlier in Week 3) that can be used as APIs to project charts

```
50
    /charts/enrollments.png
51      path( route: "charts/enrollments.png", enrollments_chart_png, name="enrollments-chart-png"),
52
53
54  ]
```



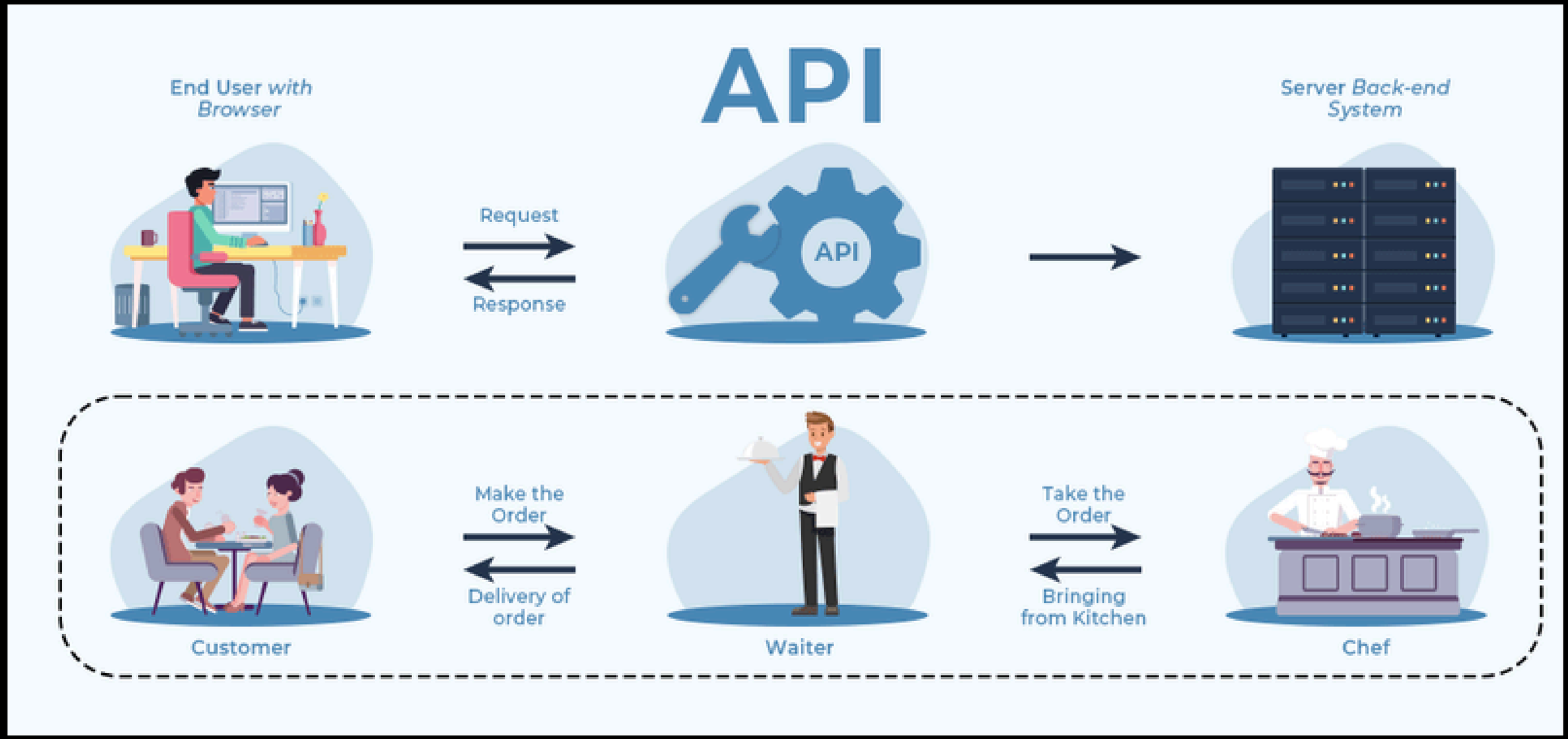
Or, this chart can directly be displayed on a webpage

```
376  # Optional: You can display the image from enrollments_chart_png() directly on a link.
377  # But if you want, you can also display it in another html page.
     /charts/enrollments/
378  class EnrollmentsChartPage(TemplateView):  2 usages
379      template_name = "students/enrollments_chart.html"
380
```



Activity time!

2. External APIs



Source: https://media.geeksforgeeks.org/wp-content/uploads/20230216170349/What-is-an-API.png

Our aim for the current week:

1. **Using External Public API (no - key required)**

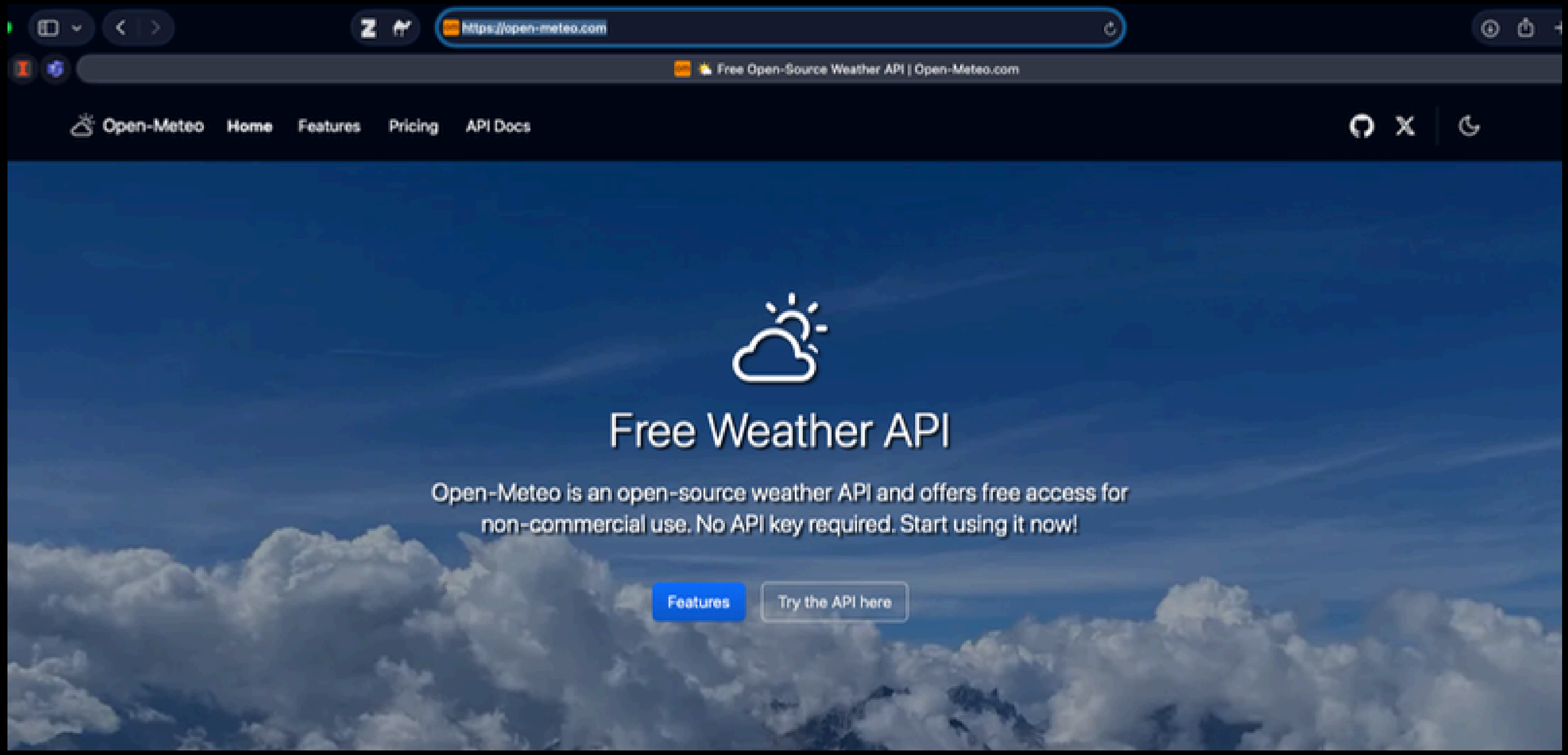
no-key required

```
r = requests.get(
    "https://api.open-meteo.com/v1/forecast",
    params={"latitude": 40.1164, "longitude": -88.2434, "current_weather": True}
)
```

Key required

```
API_KEY = os.getenv("OPENWEATHER_KEY")  # stored safely in environment
r = requests.get(
    "https://api.openweathermap.org/data/2.5/weather",
    params={"lat": 40.1164, "lon": -88.2434, "appid": API_KEY, "units": "metric"}
)
```

Open-Meteo Free API
for
Weather Data


https://open-meteo.com
Free Open-Source Weather API | Open-Meteo.com
Open-Meteo
Home
Features
Pricing
API Docs
Free Weather API
Open-Meteo is an open-source weather API and offers free access for non-commercial use. No API key required. Start using it now!
Features
Try the API here

Accurate Weather Forecasts for Any Location

Open-Meteo partners with national weather services to bring you open data with high resolution, ranging from 1 to 11 kilometres. Our powerful APIs intelligently select the most suitable weather models for your specific location, ensuring accurate and reliable forecasts.

With our user-friendly JSON API, accessing weather data has never been easier. Whether you're developing an application or seeking weather information for personal use, our APIs provide seamless integration and deliver the data you need in a simple and accessible format.

Experience the precision and convenience of Open-Meteo's Forecast API, providing comprehensive weather information worldwide. Stay informed and make informed decisions with our reliable weather forecasts.

See features | Read the docs

Forecast & Current | Last 10 Days | Historical Data

```
$ curl "https://api.open-meteo.com/v1/forecast?latitude=52.52&longitude=13.41&current=temperature_2m,wind_speed_10m&hourly=temperature_2m,relative_humidity_2m,wind_speed_10m"
```

```
{
  ...
  "current": {
    "time": "2022-01-01T15:00",
    "temperature_2m": 2.4,
    "wind_speed_10m": 11.9,
  },
  "hourly": {
    "time": ["2022-07-01T00:00","2022-07-01T01:00", ...],
    "wind_speed_10m": [3.16,3.02,3.3,3.14,3.2,2.95, ...],
    "temperature_2m": [13.7,13.3,12.8,12.3,11.8, ...],
    "relative_humidity_2m": [82,83,86,85,88,88,84,76, ...],
  }
}
```

High Resolution

Open-Meteo leverages a powerful combination of global (11 km) and mesoscale (1 km) weather models from esteemed national weather services, providing comprehensive forecasts with remarkable precision. No matter where you are in the world, you can access the most reliable and accurate weather predictions available.

Our weather data is presented in hourly resolution, allowing you to plan your activities with confidence. The initial days of the forecast benefit from localised weather models, offering highly detailed and accurate information. Subsequently, global weather models provide forecasts for up to 16 days. Through seamless integration, our APIs deliver a straightforward and reliable hourly weather forecast experience.

Rapid Updates

At Open-Meteo, we understand the importance of having the most up-to-date weather information. That's why our local weather models are updated every hour, ensuring that our forecasts reflect the latest changes in conditions, including updates from rain radars.

Our weather models rely on a wealth of real-time data, including measurements from various sources such as airplanes, buoys, radar systems, and satellites. By incorporating this diverse and comprehensive data, our numerical weather predictions provide a deeper analysis than traditional weather stations, resulting in more accurate forecasts.

80 Years Historical Data

Explore the past with our comprehensive Historical Weather API. With over 80 years of hourly weather data available at a 10 kilometre resolution, you can dive into the climate of any location. Behind the scenes, this extensive dataset, comprising 50 TB of information, enables you to access temperature records spanning eight decades in an instant.

Moreover, our 1 kilometre weather models continuously archive recent data, ensuring that you can seamlessly retrieve the latest forecasts alongside historical information from previous weeks. This functionality opens up possibilities for training machine learning applications and gaining valuable insights from the combination of present and past weather data. Discover the power of our historical weather API and unlock a treasure trove of weather information.

Open-Source

We believe in the power of open-source software. That's why the entire codebase of Open-Meteo is accessible on GitHub, released under the AGPLv3 licence. This means you can explore, use, modify, and contribute to the code.

If you wish to take it a step further, we're here to support you in setting up your own API instances. This allows you to have complete control and enjoy practically unlimited API calls, making it ideal for demanding applications like machine learning or large language models.

In addition, our data is licenced under Attribution 4.0 International (CC BY 4.0). This means you are free to share and adapt the data, even for commercial purposes. We believe in fostering an open ecosystem that encourages transparency, collaboration and innovation.

Free API

Open-Meteo offers free access to its APIs for non-commercial use, making it convenient for individuals and developers to explore and integrate weather data into their projects. The best part is that no API key, registration, or credit card is required to enjoy this service.

We trust our users to utilise the free API responsibly and kindly request appropriate credit for the data used. While there are no strict access restrictions, we encourage fair usage of the service. If you require commercial usage or anticipate exceeding 10'000 API calls per day, we recommend considering our API subscription for enhanced features and support.

Easy to Use

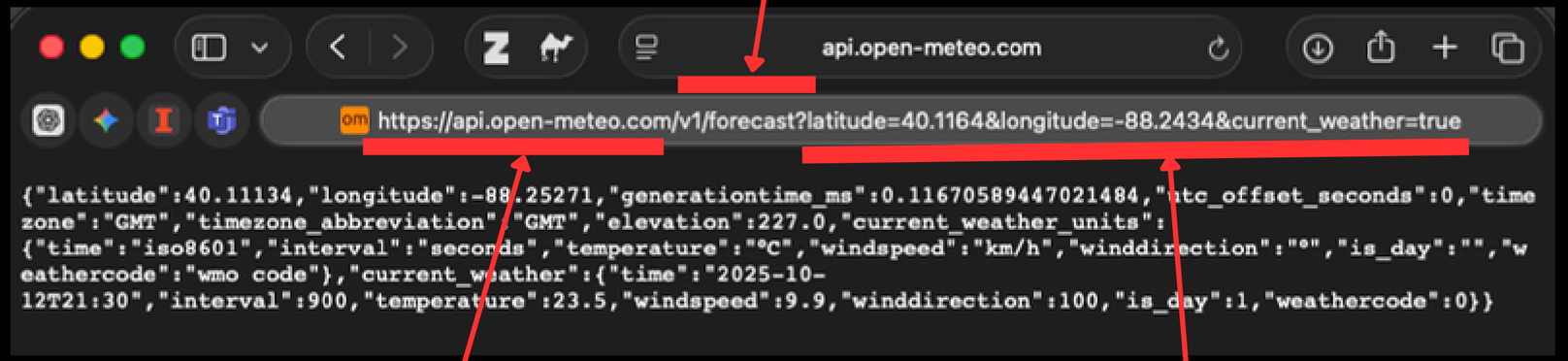
We've designed our APIs to be incredibly user-friendly. They are based on the widely adopted HTTP protocol and utilise the simplicity of JSON data format. All you need to get started is a basic understanding of geographic coordinates, making HTTP requests, and working with JSON data.

To assist you in seamlessly integrating our APIs into your projects, we provide comprehensive documentation. It includes detailed explanations of all parameters and their usage. Whether you're using Python, R, Julia, PHP, JavaScript, React, Flutter, Java, or any other programming language, our APIs are designed to work effortlessly with your application.

Endpoint: a specific table, group of linked tables/data, function that returns a specific type of output

Analogy: A specific dish at the restaurant

Eg. Burrito-bowl



API base URL

Analogy: Name of the restaurant

Eg. Qdoba

Parameters: Filtration/specific data in the selected table

Analogy: All the customizations of the bowl

Eg. a veggie burrito bowl with: 2 scoops of guacamole

	Parameter 1: Column Name 1	**Parameter 2: Column Name 2**
Attribute 1:	1.1	1.2
Attribute 2:	2.1	2.2

	Parameter 1: Burrito-Bowl	Parameter 2: Ice-Cream
Attribute 1.1: type1	Veg (100 to 200 gm)	Cone (100 to 200 gm)
Attribute 1.2: type2	Non-veg (100 to 200 gm)	cup (100 to 200 ml)

www.chipotle.com/?burrito-bowl=veg&ice-cream=cone

Is this a GET request or a POST request?

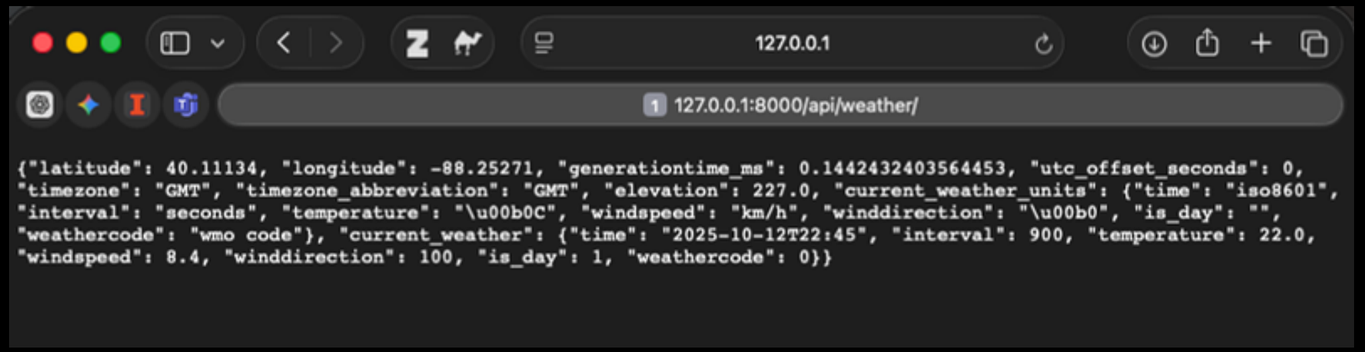
P.S. Like we covered in last week, both the requests gives us a response from the server.

- Django will raise an exception if data to be returned to html is not a dictionary in JsonResponse(). And by default, safe=True for that.
- So safe = False simply means that the data I'm sending can be a list or a dictionary or any other format.

```python
/api/weather/
class WeatherNow(View):  2 usages
    """
    Call Open-Meteo (keyless) and return just the bits we need.
    """
    def get(self, request):

        # Step 1: Prepare parameters for the API request
        params = {
            # Champaign, IL
            "latitude": 40.1164,        # Champaign, IL latitude
            "longitude": -88.2434,      # Champaign, IL longitude
            "current_weather": True,    # Ask Open-Meteo for current conditions
        }

        try:
            output_raw_all = requests.get( url: "https://api.open-meteo.com/v1/forecast",
                                params=params, timeout=5)

            output_raw_all.raise_for_status()

            output_polished_all = output_raw_all.json()
            return JsonResponse(output_polished_all, safe=False)


        except requests.exceptions.RequestException as e:

            return JsonResponse( data: {"ok": False, "error": str(e)}, status=502)
```

This returns the entire output as is.

Output 1.




```
{
  "latitude": 40.11134,
  "longitude": -88.25271,
  "generationtime_ms": 0.144,
  "utc_offset_seconds": 0,
  "timezone": "GMT",
  "timezone_abbreviation": "GMT",
  "elevation": 227.0,

  "current_weather_units": {
    "time": "iso8601",
    "interval": "seconds",
    "temperature": "°C",
    "windspeed": "km/h",
    "winddirection": "°",
    "is_day": "",
    "weathercode": "wmo code"
  },

  "current_weather": {
    "time": "2025-10-12T22:45",
    "interval": 900,
    "temperature": 22.0,
    "windspeed": 8.4,
    "winddirection": 100,
    "is_day": 1,
    "weathercode": 0
  }
}
```

Code Range	Meaning	Example
100 - 199	Informational	Keep going, still processing.
200 - 299	Success	Here’s what you asked for.
300 - 399	Redirection	Go here instead
400 - 499	Client Error	You did something wrong.
500 - 599	Server Error	I (the server) messed up.

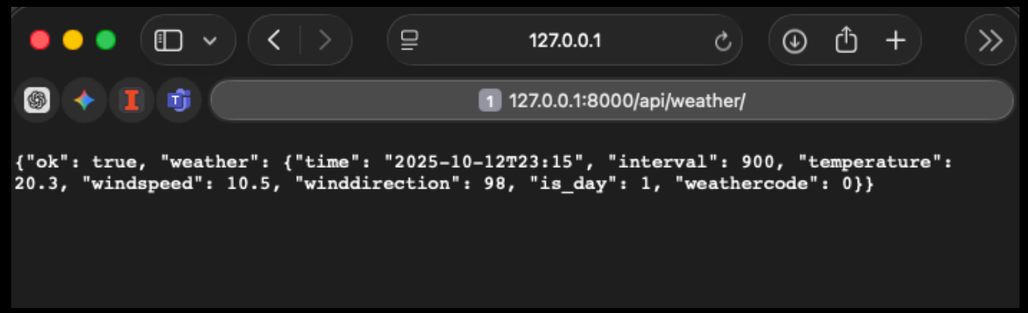
Output 2.

If I set:

output_polished_cw_only = output_polished_all.get (“current_weather”, {})

if only selects the current_weather key and its values for sending back.

```
output_polished_all = output_raw_all.json()

# Step 5: Extract just the values of "current_weather" key from the JSON
# If missing, return an empty dictionary instead of crashing
## Expected raw data:
##   {
##          "key 1":
##                 {
#                     key 1.1: "value 1.1",
#                     key 1.2: "value 1.2",
#                 },
#
#          "current_weather":                                    <---- This is what we want
#                 {
#                     "time": "2025-10-12T22:45",
#                     "interval": 900,
#                     "temperature": 22.0,
#                     "windspeed": 8.4,
#                     "winddirection": 100,
#                     "is_day": 1,
#                     "weathercode": 0
#                  }
##    }
output_polished_cw_only = output_polished_all.get("current_weather", {})


# Step 6: Return success response with simplified data
# The browser (or JS frontend) can consume this JSON directly
return JsonResponse({"ok": True, "weather": output_polished_cw_only})
```



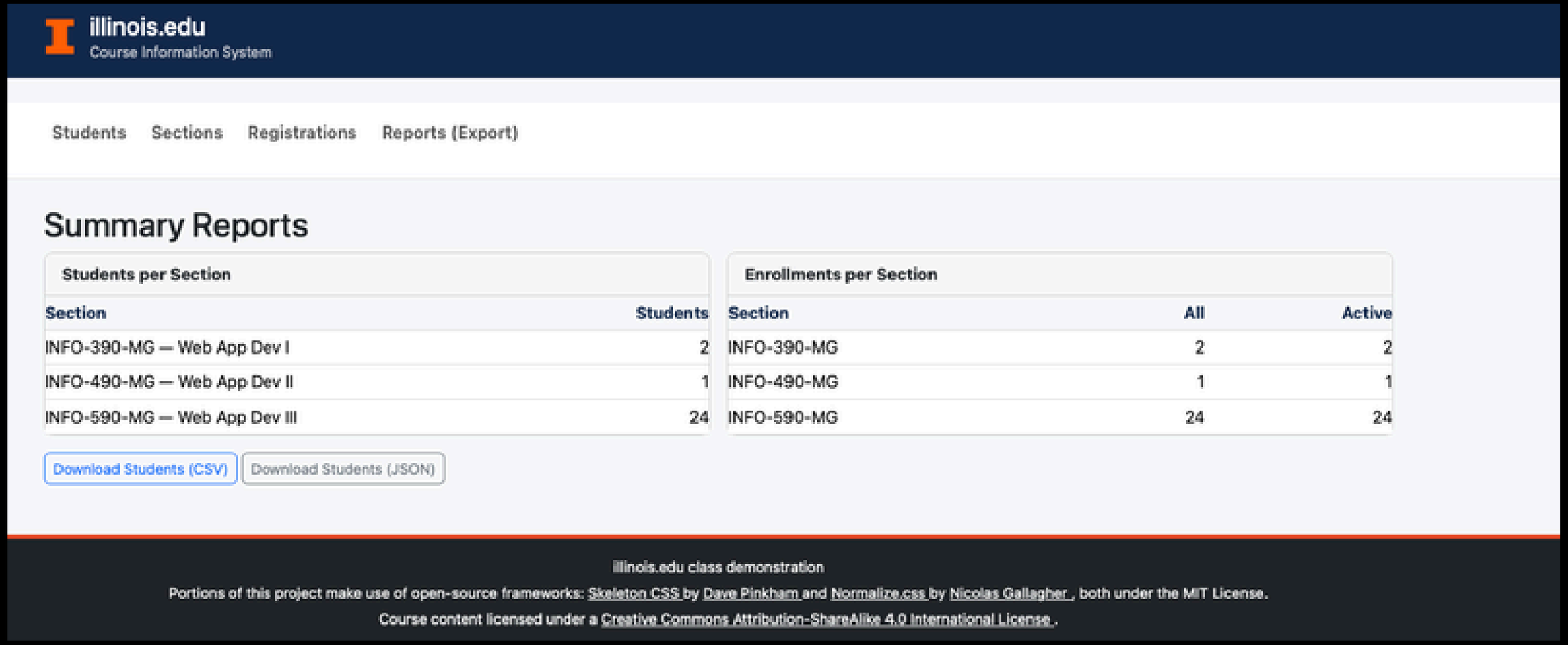

```
{
  "ok": true,
  "weather": {
    "time": "2025-10-12T23:15",
    "interval": 900,
    "temperature": 20.3,
    "windspeed": 10.5,
    "winddirection": 98,
    "is_day": 1,
    "weathercode": 0
  }
}
```

Activity time!

3. Export (CSV / JSON)

a. csv export
b. json export

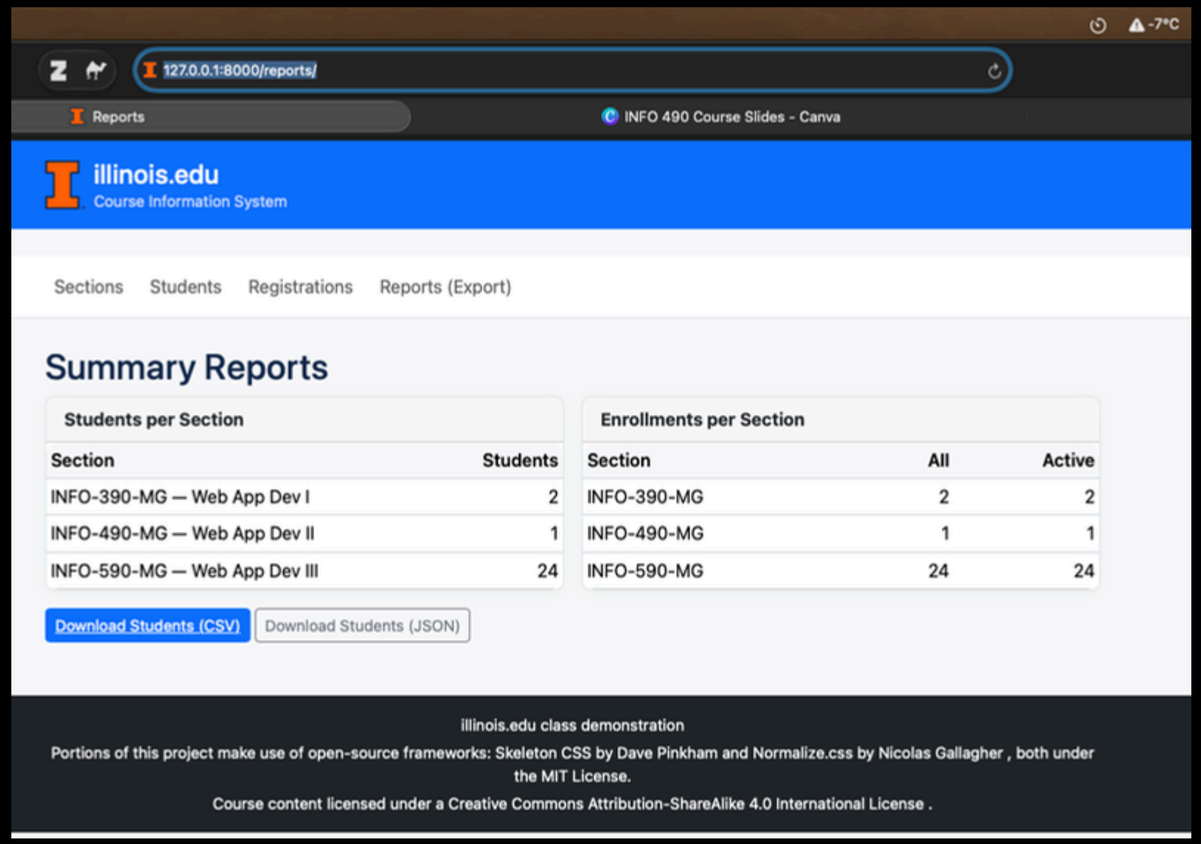
Let’s first create a
new report section.



But, the download
links doesn’t work yet.

3.a CSV Export

The download link will now work after this sub-section.



1. Create filename

- Generate a unique, time-stamped filename so downloads don’t overwrite each other.

- datetime.now().strftime()
- "students_2025-10-12_17-35.csv"

2. Prepare HttpResponse

- Tell Django we’re sending a CSV file, not HTML.

- HttpResponse(content_type="text/csv")
- Response object (empty for now).

3. Add Download Header

- Instruct the browser to download instead of display it inline.

- response["Content-Disposition"]
- File download prompt when sent.

4. Create CSV Writer

- Make the response behave like a writable file object.

- csv.writer(response)
- Writer ready to fill rows.
- Analogically, imagine that you are calling a friend who is a writer. And that friend is now ready to write, and is just waiting for your instructions.

5. Query Data

- Fetch only needed fields from the database, optimized for output.
- Student.objects.values_list(...)

6. Write Header + Rows

- Add header (column names) row + all student data.
- writer.writerow([...])
- Response now filled with CSV text.
- Analogically, now you gave your friend a page to copy. Now your friend reads one line, writes that line, and then continues this style of copying and writing.

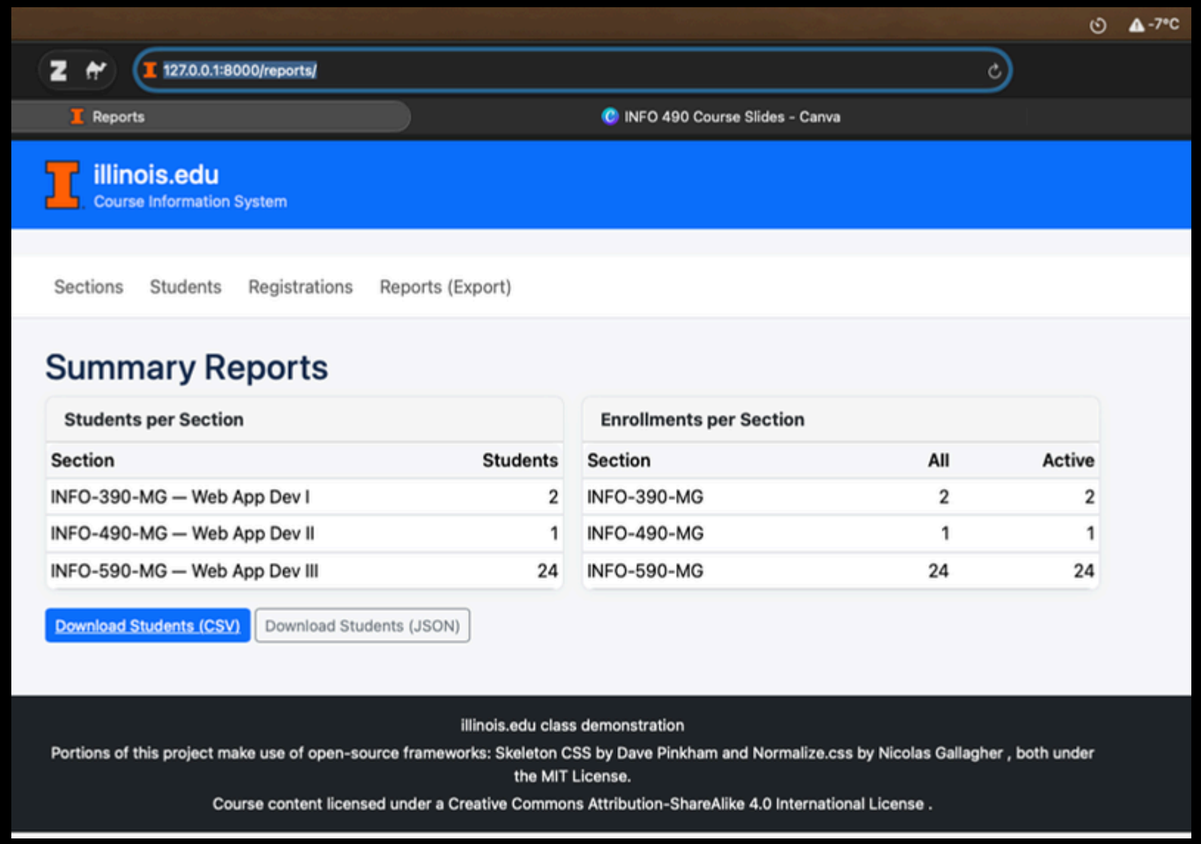
7. Return Response

- Django sends it to the browser.
- return response
- Browser downloads CSV file. And the file can be downloaded by the user with a click

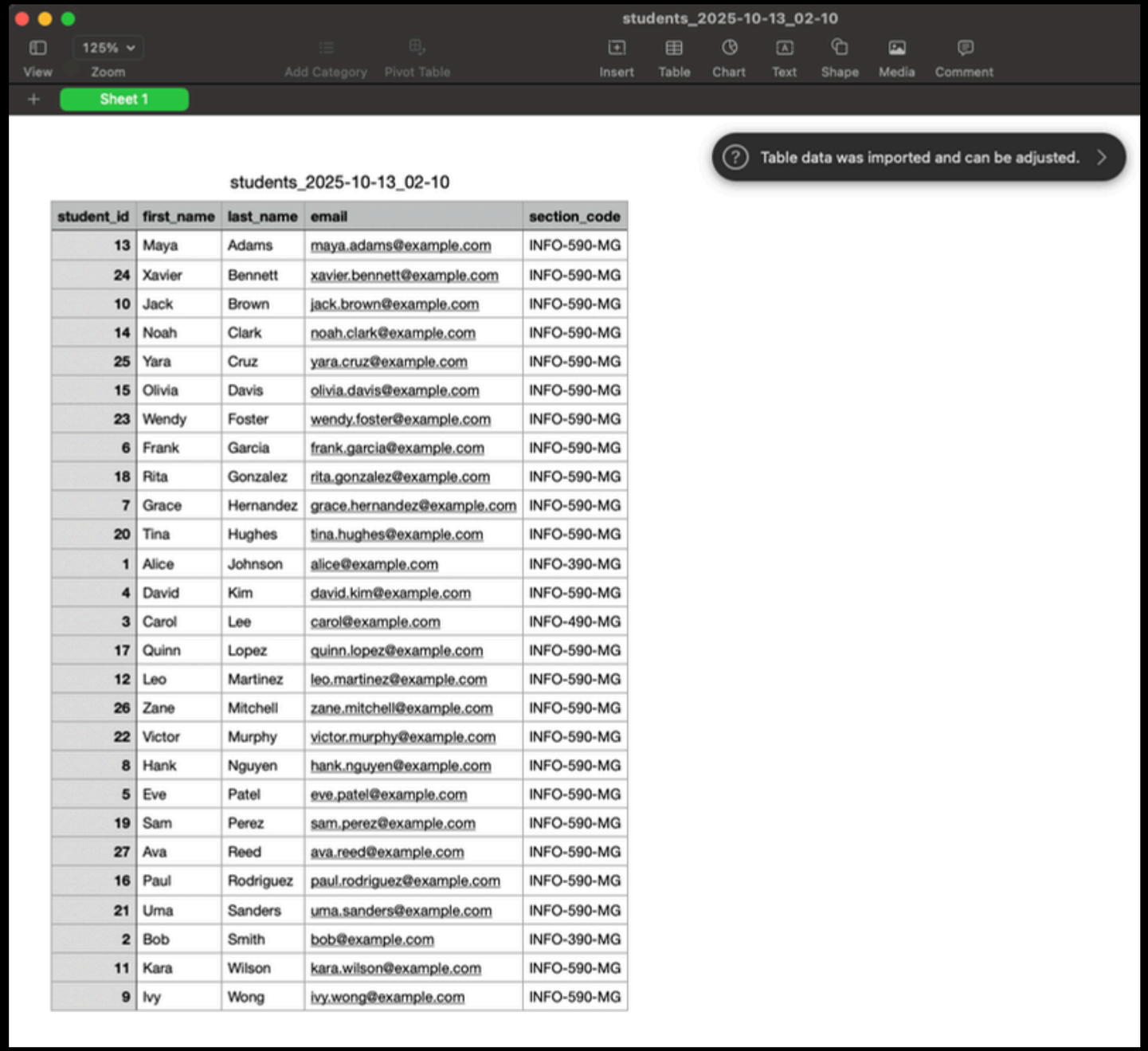
Once you are done integrating the scripts for this section, try clicking the

Download Students (CSV) button.

It gets highlighted in blue color.



You can now download the data as a csv



students_2025-10-13_02-10

student_id	first_name	last_name	email	section_code
13	Maya	Adams	maya.adams@example.com	INFO-590-MG
24	Xavier	Bennett	xavier.bennett@example.com	INFO-590-MG
10	Jack	Brown	jack.brown@example.com	INFO-590-MG
14	Noah	Clark	noah.clark@example.com	INFO-590-MG
25	Yara	Cruz	yara.cruz@example.com	INFO-590-MG
15	Olivia	Davis	olivia.davis@example.com	INFO-590-MG
23	Wendy	Foster	wendy.foster@example.com	INFO-590-MG
6	Frank	Garcia	frank.garcia@example.com	INFO-590-MG
18	Rita	Gonzalez	rita.gonzalez@example.com	INFO-590-MG
7	Grace	Hernandez	grace.hernandez@example.com	INFO-590-MG
20	Tina	Hughes	tina.hughes@example.com	INFO-590-MG
1	Alice	Johnson	alice@example.com	INFO-390-MG
4	David	Kim	david.kim@example.com	INFO-590-MG
3	Carol	Lee	carol@example.com	INFO-490-MG
17	Quinn	Lopez	quinn.lopez@example.com	INFO-590-MG
12	Leo	Martinez	leo.martinez@example.com	INFO-590-MG
26	Zane	Mitchell	zane.mitchell@example.com	INFO-590-MG
22	Victor	Murphy	victor.murphy@example.com	INFO-590-MG
8	Hank	Nguyen	hank.nguyen@example.com	INFO-590-MG
5	Eve	Patel	eve.patel@example.com	INFO-590-MG
19	Sam	Perez	sam.perez@example.com	INFO-590-MG
27	Ava	Reed	ava.reed@example.com	INFO-590-MG
16	Paul	Rodriguez	paul.rodriguez@example.com	INFO-590-MG
21	Uma	Sanders	uma.sanders@example.com	INFO-590-MG
2	Bob	Smith	bob@example.com	INFO-390-MG
11	Kara	Wilson	kara.wilson@example.com	INFO-590-MG
9	Ivy	Wong	ivy.wong@example.com	INFO-590-MG

3.b JSON Export

Follows similar procedure as sub-section **3.a** but with minute changes to save file in json format.

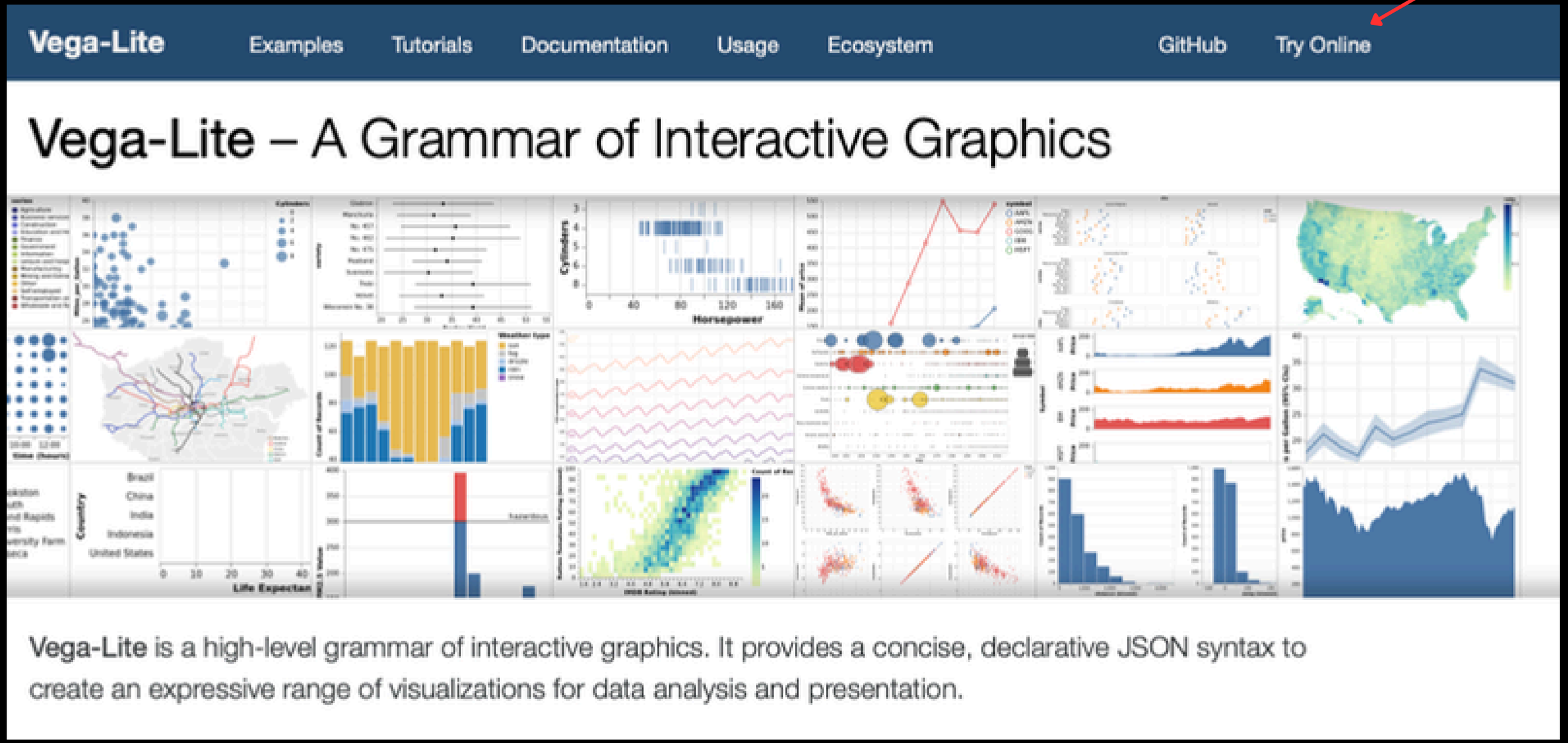
You can open this format in google chrome, text app, or anything that can read JSON

Activity time!

4. Advanced Charts: Vega-Lite

.

https://vega.github.io/vega-lite/



Our API data also looks like this!

The Data

Let's say you have a tabular data set with a categorical variable in the first column `a` and a numerical variable in the second column `b`.

a	b
C	2
C	7
C	4
D	1
D	2
D	6
E	8
E	4
E	7

We can represent this data as a JSON array in which each row is an object in the array.

```
[
  {"a": "C", "b": 2},
  {"a": "C", "b": 7},
  {"a": "C", "b": 4},
  {"a": "D", "b": 1},
  {"a": "D", "b": 2},
  {"a": "D", "b": 6},
  {"a": "E", "b": 8},
  {"a": "E", "b": 4},
  {"a": "E", "b": 7}
]
```

Step 1: Put it inside a dictionary which has the following nested key value pair {key: {“value”}}:

```
{
 ”data”: {”values: []”}
}
```

To visualize this data with Vega-Lite, we can add it directly to the `data` property in a Vega-Lite specification.

```
{
  "data": {
    "values": [
      {"a": "C", "b": 2},
      {"a": "C", "b": 7},
      {"a": "C", "b": 4},
      {"a": "D", "b": 1},
      {"a": "D", "b": 2},
      {"a": "D", "b": 6},
      {"a": "E", "b": 8},
      {"a": "E", "b": 4},
      {"a": "E", "b": 7}
    ]
  }
}
```

Step 2: Define what kind of graphs we want using Encoding

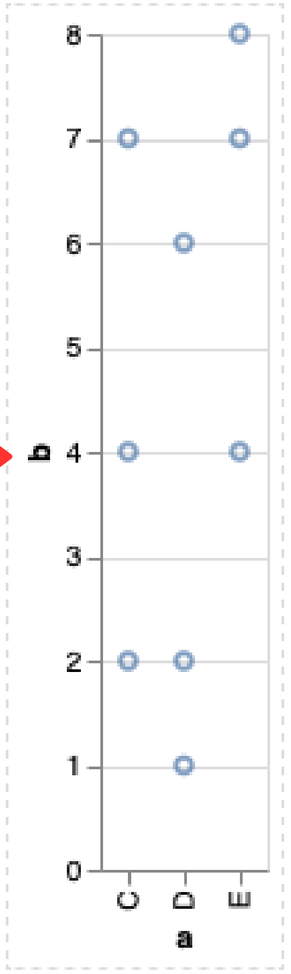
```
{
  "data": {
    "values": [
      {"a": "C", "b": 2}, {"a": "C", "b": 7}, {"a": "C", "b": 4},
      {"a": "D", "b": 1}, {"a": "D", "b": 2}, {"a": "D", "b": 6},
      {"a": "E", "b": 8}, {"a": "E", "b": 4}, {"a": "E", "b": 7}
    ]
  },
  "mark": "point",
  "encoding": {}
}
```

```
...
"encoding": {
  "x": {"field": "a", "type": "nominal"}
}
...
```

Step 3: Combine everything together

a	b
C	2
C	7
C	4
D	1
D	2
D	6
E	8
E	4
E	7

```
{
  "data": {
    "values": [
      {"a": "C", "b": 2}, {"a": "C", "b": 7}, {"a": "C", "b": 4},
      {"a": "D", "b": 1}, {"a": "D", "b": 2}, {"a": "D", "b": 6},
      {"a": "E", "b": 8}, {"a": "E", "b": 4}, {"a": "E", "b": 7}
    ]
  },
  "mark": "point",
  "encoding": {
    "x": {"field": "a", "type": "nominal"},
    "y": {"field": "b", "type": "quantitative"}
  }
}
```



Open in Vega Editor

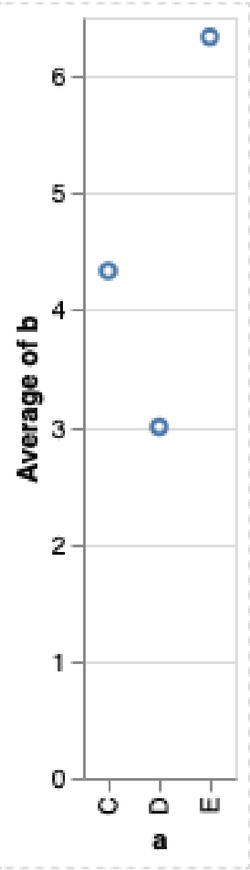
Step 4: You can also apply operations like aggregations

a	b
C	2
C	7
C	4
D	1
D	2
D	6
E	8
E	4
E	7

```
{
  "data": {
    "values": [
      {"a": "C", "b": 2}, {"a": "C", "b": 7}, {"a": "C", "b": 4},
      {"a": "D", "b": 1}, {"a": "D", "b": 2}, {"a": "D", "b": 6},
      {"a": "E", "b": 8}, {"a": "E", "b": 4}, {"a": "E", "b": 7}
    ]
  },
  "mark": "point",
  "encoding": {
    "x": {"field": "a", "type": "nominal"},
    "y": {"aggregate": "average", "field": "b", "type": "quantitative"}
  }
}
```

Data Transformation: Aggregation

Vega-Lite also supports data transformation such as aggregation. By adding `"aggregate": "average"` to the definition of the `y` channel, we can see the average value of `a` in each category. For example, the average value of category `D` is `(1 + 2 + 6)/3 = 9/3 = 3`.



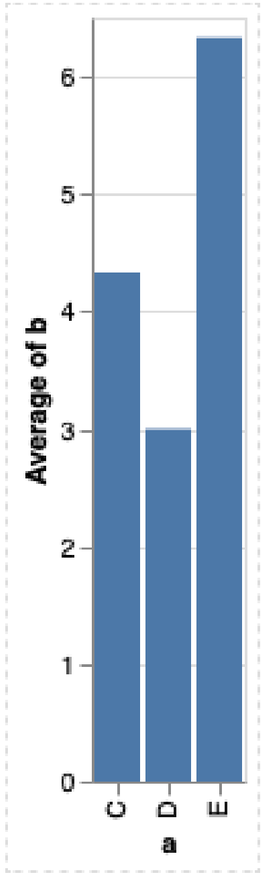
Open in Vega Editor

Step 5: Or change the chart type simply by replacing {“mark”: “point”} with {“mark”: “bar”}

```
- "mark": "point"
+ "mark": "bar"
```

a	b
C	2
C	7
C	4
D	1
D	2
D	6
E	8
E	4
E	7

```
{
  "data": {
    "values": [
      {"a": "C", "b": 2}, {"a": "C", "b": 7}, {"a": "C", "b": 4},
      {"a": "D", "b": 1}, {"a": "D", "b": 2}, {"a": "D", "b": 6},
      {"a": "E", "b": 8}, {"a": "E", "b": 4}, {"a": "E", "b": 7}
    ]
  },
  "mark": "bar",
  "encoding": {
    "x": {"field": "a", "type": "nominal"},
    "y": {"aggregate": "average", "field": "b", "type": "quantitative"}
  }
}
```



Open in Vega Editor

Step 6: **Publishing is very simple. You only need to take care of these 5 important steps**

```html
<!doctype html>
<html>
  <head>
    <title>Vega-Lite Bar Chart</title>
    <meta charset="utf-8" />

    <script src="https://cdn.jsdelivr.net/npm/vega@6.2.0"></script>
    <script src="https://cdn.jsdelivr.net/npm/vega-lite@6.4.1"></script>
    <script src="https://cdn.jsdelivr.net/npm/vega-embed@7.0.2"></script>

    <style media="screen">
      /* Add space between Vega-Embed links  */
      .vega-actions a {
        margin-right: 5px;
      }
    </style>
  </head>
  <body>
    <h1>Template for Embedding Vega-Lite Visualization</h1>
    <!-- Container for the visualization -->
    <div id="vis"></div>

    <script>
      // Assign the specification to a local variable vlSpec.
      var vlSpec = {
        $schema: 'https://vega.github.io/schema/vega-lite/v6.json',
        data: {
          values: [
            {a: 'C', b: 2},
            {a: 'C', b: 7},
            {a: 'C', b: 4},
            {a: 'D', b: 1},
            {a: 'D', b: 2},
            {a: 'D', b: 6},
            {a: 'E', b: 8},
            {a: 'E', b: 4},
            {a: 'E', b: 7},
          ],
        },
        mark: 'bar',
        encoding: {
          y: {field: 'a', type: 'nominal'},
          x: {
            aggregate: 'average',
            field: 'b',
            type: 'quantitative',
            axis: {
              title: 'Average of b',
            },
          },
        },
      };

      // Embed the visualization in the container with id `vis`
      vegaEmbed('#vis', vlSpec);
    </script>
  </body>
</html>
```

Step 6.1: Reference the <script> having important Vega-lite libraries

```html
<!doctype html>
<html>
  <head>
    <title>Vega-Lite Bar Chart</title>
    <meta charset="utf-8" />

    <script src="https://cdn.jsdelivr.net/npm/vega@6.2.0"></script>
    <script src="https://cdn.jsdelivr.net/npm/vega-lite@6.4.1"></script>
    <script src="https://cdn.jsdelivr.net/npm/vega-embed@7.0.2"></script>

    <style media="screen">
      /* Add space between Vega-Embed links  */
      .vega-actions a {
        margin-right: 5px;
      }
    </style>
  </head>
  <body>
    <h1>Template for Embedding Vega-Lite Visualization</h1>
    <!-- Container for the visualization -->
    <div id="vis"></div>

    <script>
      // Assign the specification to a local variable vlSpec.
      var vlSpec = {
        $schema: 'https://vega.github.io/schema/vega-lite/v6.json',
        data: {
          values: [
            {a: 'C', b: 2},
            {a: 'C', b: 7},
            {a: 'C', b: 4},
            {a: 'D', b: 1},
            {a: 'D', b: 2},
            {a: 'D', b: 6},
            {a: 'E', b: 8},
            {a: 'E', b: 4},
            {a: 'E', b: 7},
          ],
        },
        mark: 'bar',
        encoding: {
          y: {field: 'a', type: 'nominal'},
          x: {
            aggregate: 'average',
            field: 'b',
            type: 'quantitative',
            axis: {
              title: 'Average of b',
            },
          },
        },
      };

      // Embed the visualization in the container with id `vis`
      vegaEmbed('#vis', vlSpec);
    </script>
  </body>
</html>
```

Step 6.2: Adds styling / css

```
<!doctype html>
<html>
  <head>
    <title>Vega-Lite Bar Chart</title>
    <meta charset="utf-8" />

    <script src="https://cdn.jsdelivr.net/npm/vega@6.2.0"></script>
    <script src="https://cdn.jsdelivr.net/npm/vega-lite@6.4.1"></script>
    <script src="https://cdn.jsdelivr.net/npm/vega-embed@7.0.2"></script>

    <style media="screen">
      /* Add space between Vega-Embed links  */
      .vega-actions a {
        margin-right: 5px;
      }
    </style>
  </head>
  <body>
    <h1>Template for Embedding Vega-Lite Visualization</h1>
    <!-- Container for the visualization -->
    <div id="vis"></div>

    <script>
      // Assign the specification to a local variable vlSpec.
      var vlSpec = {
        $schema: 'https://vega.github.io/schema/vega-lite/v6.json',
        data: {
          values: [
            {a: 'C', b: 2},
            {a: 'C', b: 7},
            {a: 'C', b: 4},
            {a: 'D', b: 1},
            {a: 'D', b: 2},
            {a: 'D', b: 6},
            {a: 'E', b: 8},
            {a: 'E', b: 4},
            {a: 'E', b: 7},
          ],
        },
        mark: 'bar',
        encoding: {
          y: {field: 'a', type: 'nominal'},
          x: {
            aggregate: 'average',
            field: 'b',
            type: 'quantitative',
            axis: {
              title: 'Average of b',
            },
          },
        },
      };

      // Embed the visualization in the container with id `vis`
      vegaEmbed('#vis', vlSpec);
    </script>
  </body>
</html>
```

Step 6.3: This is the container where your visualization will be displayed.

- Notice that the id mentioned here is "vis". We'll use this in step 6.6.
- You can name it whatever you want.

```
<!doctype html>
<html>
  <head>
    <title>Vega-Lite Bar Chart</title>
    <meta charset="utf-8" />

    <script src="https://cdn.jsdelivr.net/npm/vega@6.2.0"></script>
    <script src="https://cdn.jsdelivr.net/npm/vega-lite@6.4.1"></script>
    <script src="https://cdn.jsdelivr.net/npm/vega-embed@7.0.2"></script>

    <style media="screen">
      /* Add space between Vega-Embed links  */
      .vega-actions a {
        margin-right: 5px;
      }
    </style>
  </head>
  <body>
    <h1>Template for Embedding Vega-Lite Visualization</h1>
    <!-- Container for the visualization -->
    <div id="vis"></div>

    <script>
      // Assign the specification to a local variable vlSpec.
      var vlSpec = {
        $schema: 'https://vega.github.io/schema/vega-lite/v6.json',
        data: {
          values: [
            {a: 'C', b: 2},
            {a: 'C', b: 7},
            {a: 'C', b: 4},
            {a: 'D', b: 1},
            {a: 'D', b: 2},
            {a: 'D', b: 6},
            {a: 'E', b: 8},
            {a: 'E', b: 4},
            {a: 'E', b: 7},
          ],
        },
        mark: 'bar',
        encoding: {
          y: {field: 'a', type: 'nominal'},
          x: {
            aggregate: 'average',
            field: 'b',
            type: 'quantitative',
            axis: {
              title: 'Average of b',
            },
          },
        },
      };

      // Embed the visualization in the container with id `vis`
      vegaEmbed('#vis', vlSpec);
    </script>
  </body>
</html>
```

Step 6.4: All of our code goes inside the <script> tag:

```
<script>

    var vlSpec = {};

    vegaEmbed();

</script>
```

```
<!doctype html>
<html>
  <head>
    <title>Vega-Lite Bar Chart</title>
    <meta charset="utf-8" />

    <script src="https://cdn.jsdelivr.net/npm/vega@6.2.0"></script>
    <script src="https://cdn.jsdelivr.net/npm/vega-lite@6.4.1"></script>
    <script src="https://cdn.jsdelivr.net/npm/vega-embed@7.0.2"></script>

    <style media="screen">
      /* Add space between Vega-Embed links  */
      .vega-actions a {
        margin-right: 5px;
      }
    </style>
  </head>
  <body>
    <h1>Template for Embedding Vega-Lite Visualization</h1>
    <!-- Container for the visualization -->
    <div id="vis"></div>

    <script>
      // Assign the specification to a local variable vlSpec.
      var vlSpec = {
        $schema: 'https://vega.github.io/schema/vega-lite/v6.json',
        data: {
          values: [
            {a: 'C', b: 2},
            {a: 'C', b: 7},
            {a: 'C', b: 4},
            {a: 'D', b: 1},
            {a: 'D', b: 2},
            {a: 'D', b: 6},
            {a: 'E', b: 8},
            {a: 'E', b: 4},
            {a: 'E', b: 7},
          ],
        },
        mark: 'bar',
        encoding: {
          y: {field: 'a', type: 'nominal'},
          x: {
            aggregate: 'average',
            field: 'b',
            type: 'quantitative',
            axis: {
              title: 'Average of b',
            },
          },
        },
      };

      // Embed the visualization in the container with id `vis`
      vegaEmbed('#vis', vlSpec);
    </script>
  </body>
</html>
```

Step 6.5: This is the heart of the visualization. You paste all of your code here.

- You save it in a variable called “vlSpec”. We’ll use this in step 6.6. to embed our visualization to a specific div container.
- You can name it whatever you want.

```
<!doctype html>
<html>
  <head>
    <title>Vega-Lite Bar Chart</title>
    <meta charset="utf-8" />

    <script src="https://cdn.jsdelivr.net/npm/vega@6.2.0"></script>
    <script src="https://cdn.jsdelivr.net/npm/vega-lite@6.4.1"></script>
    <script src="https://cdn.jsdelivr.net/npm/vega-embed@7.0.2"></script>

    <style media="screen">
      /* Add space between Vega-Embed links  */
      .vega-actions a {
        margin-right: 5px;
      }
    </style>
  </head>
  <body>
    <h1>Template for Embedding Vega-Lite Visualization</h1>
    <!-- Container for the visualization -->
    <div id="vis"></div>

    <script>
      // Assign the specification to a local variable vlSpec.
      var vlSpec = {
        $schema: 'https://vega.github.io/schema/vega-lite/v6.json',
        data: {
          values: [
            {a: 'C', b: 2},
            {a: 'C', b: 7},
            {a: 'C', b: 4},
            {a: 'D', b: 1},
            {a: 'D', b: 2},
            {a: 'D', b: 6},
            {a: 'E', b: 8},
            {a: 'E', b: 4},
            {a: 'E', b: 7},
          ],
        },
        mark: 'bar',
        encoding: {
          y: {field: 'a', type: 'nominal'},
          x: {
            aggregate: 'average',
            field: 'b',
            type: 'quantitative',
            axis: {
              title: 'Average of b',
            },
          },
        },
      };

      // Embed the visualization in the container with id `vis`
      vegaEmbed('#vis', vlSpec);
    </script>
  </body>
</html>
```

Step 6.6: This is where you'll send the code you wrote inside "vlSpec" to a container with the id "vis"

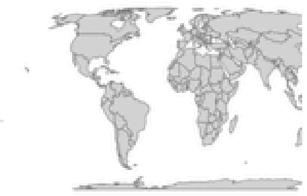
vegaEmbed(Step 6.3 name, Step 6.6 name)

vegaEmbed('#vis', vlSpec)

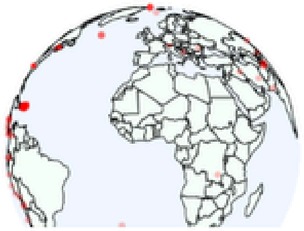
```
<!doctype html>
<html>
  <head>
    <title>Vega-Lite Bar Chart</title>
    <meta charset="utf-8" />

    <script src="https://cdn.jsdelivr.net/npm/vega@6.2.0"></script>
    <script src="https://cdn.jsdelivr.net/npm/vega-lite@6.4.1"></script>
    <script src="https://cdn.jsdelivr.net/npm/vega-embed@7.0.2"></script>

    <style media="screen">
      /* Add space between Vega-Embed links  */
      .vega-actions a {
        margin-right: 5px;
      }
    </style>
  </head>
  <body>
    <h1>Template for Embedding Vega-Lite Visualization</h1>
    <!-- Container for the visualization -->
    <div id="vis"></div>

    <script>
      // Assign the specification to a local variable vlSpec.
      var vlSpec = {
        $schema: 'https://vega.github.io/schema/vega-lite/v6.json',
        data: {
          values: [
            {a: 'C', b: 2},
            {a: 'C', b: 7},
            {a: 'C', b: 4},
            {a: 'D', b: 1},
            {a: 'D', b: 2},
            {a: 'D', b: 6},
            {a: 'E', b: 8},
            {a: 'E', b: 4},
            {a: 'E', b: 7},
          ],
        },
        mark: 'bar',
        encoding: {
          y: {field: 'a', type: 'nominal'},
          x: {
            aggregate: 'average',
            field: 'b',
            type: 'quantitative',
            axis: {
              title: 'Average of b',
            },
          },
        },
      };

      // Embed the visualization in the container with id `vis`
      vegaEmbed('#vis', vlSpec);
    </script>
  </body>
</html>
```

https://vega.github.io/vega-lite/examples

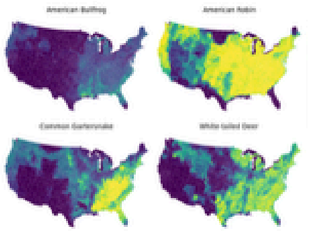
London Tube Lines



Projection explorer



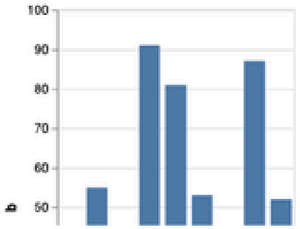
Earthquakes Example



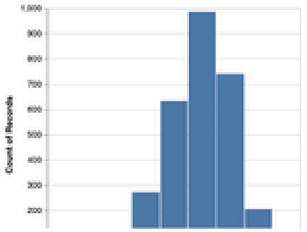
Faceted County-Level Species Habitat Maps

Interactive

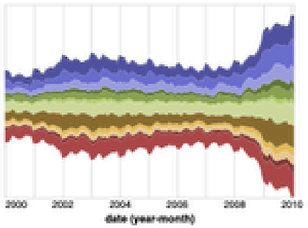
Interactive Charts



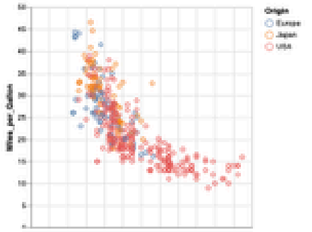
Bar Chart with Highlighting on Hover and Selection on Click



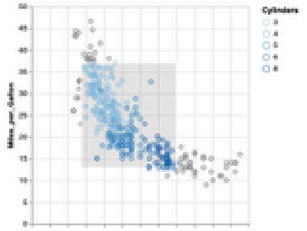
Histogram with Full-Height Hover Targets for Tooltip



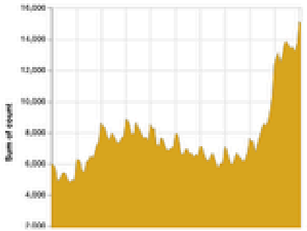
Interactive Legend



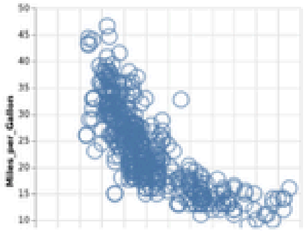
Scatterplot with External Links and Tooltips



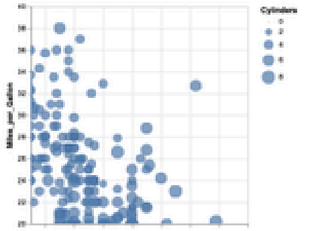
Rectangular Brush



Area Chart with Rectangular Brush



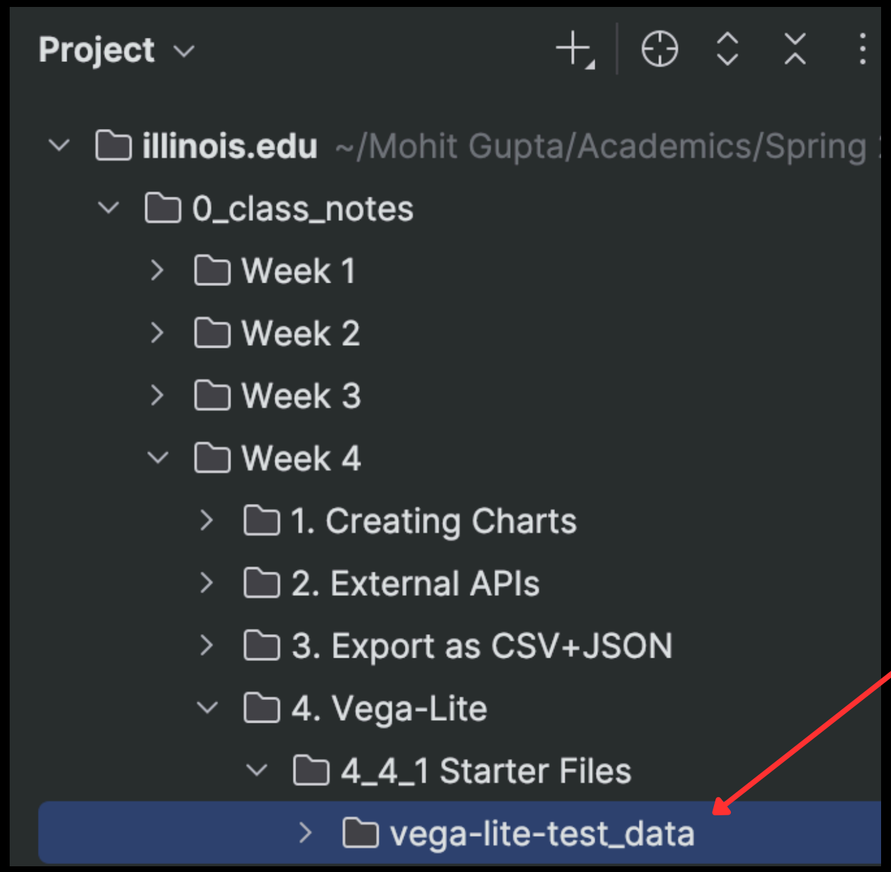
Paintbrush Highlight

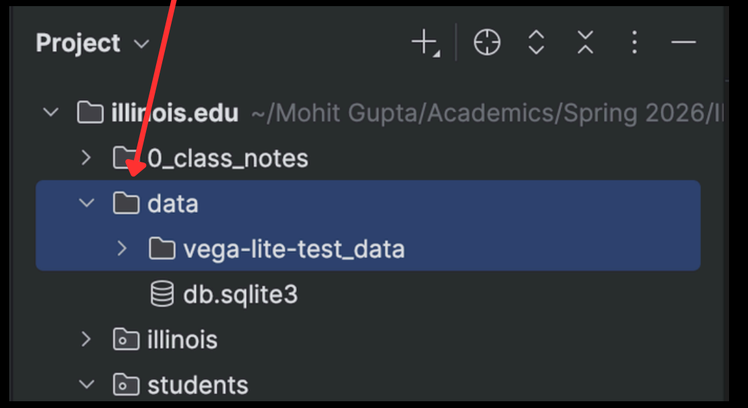


Scatterplot Pan & Zoom

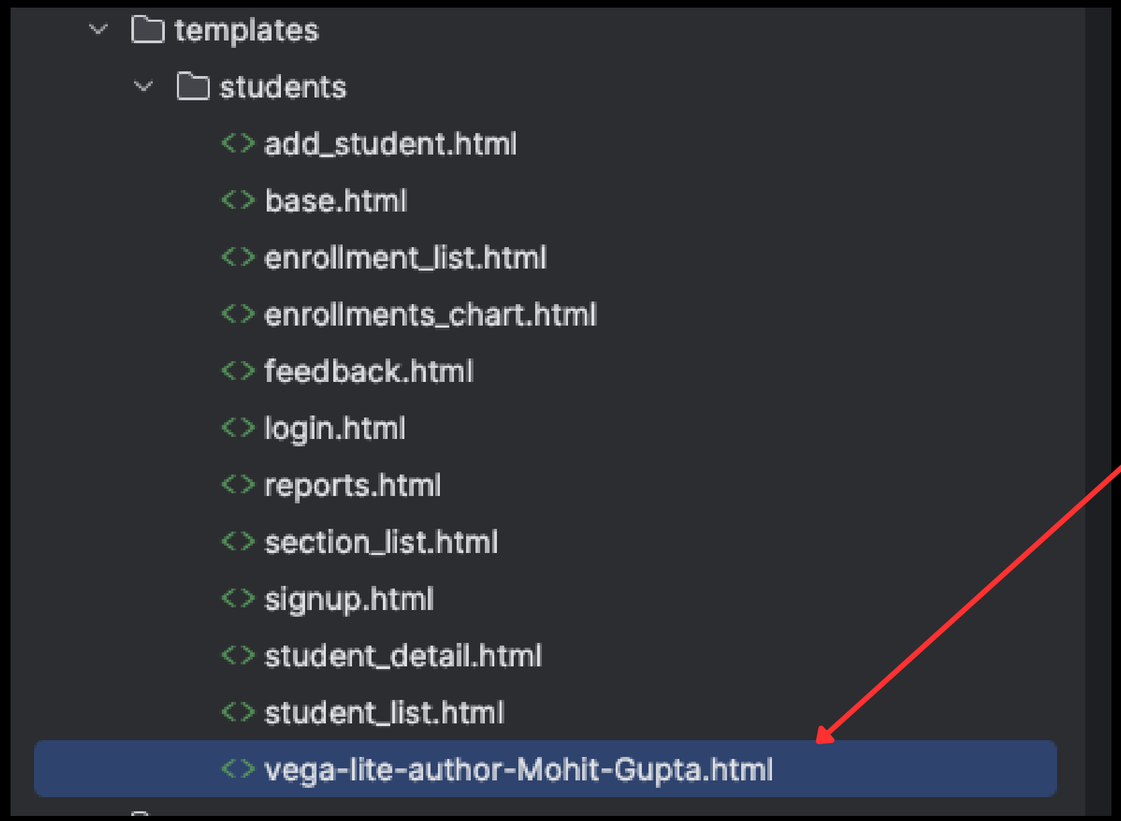
Sharing something I built in the non-ChatGPT era in 2021.

Step 1:
Paste this folder in the illinois/data folder.



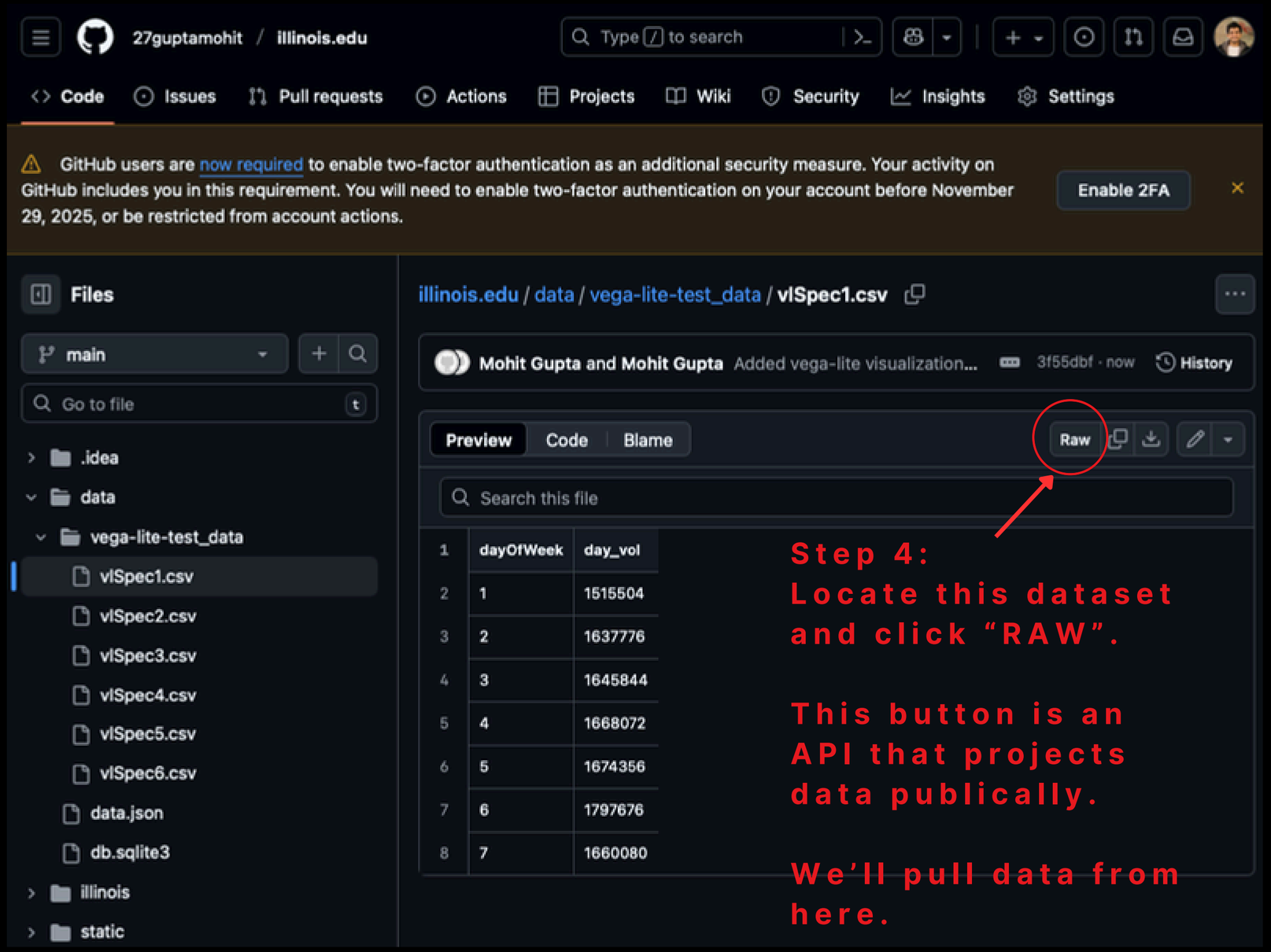


Step 2:
Paste the html in the “templates/students” folder



Step 3:
Open our github folder and locate this dataset.

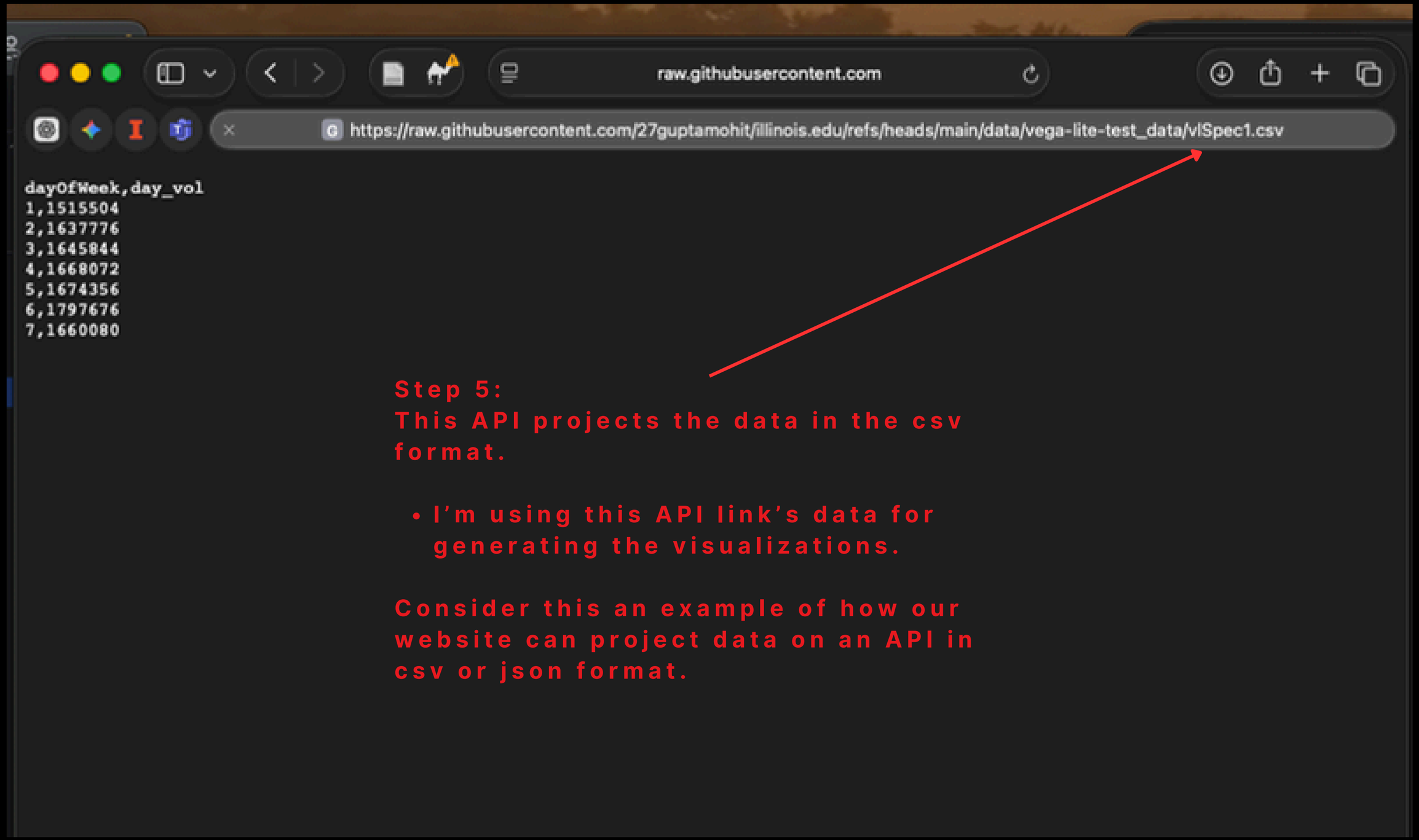
https://github.com/27guptamohit/illinois.edu/blob/main/data/vega-lite-test_data/vlSpec1.csv



Step 4:
Locate this dataset and click "RAW".

This button is an API that projects data publically.

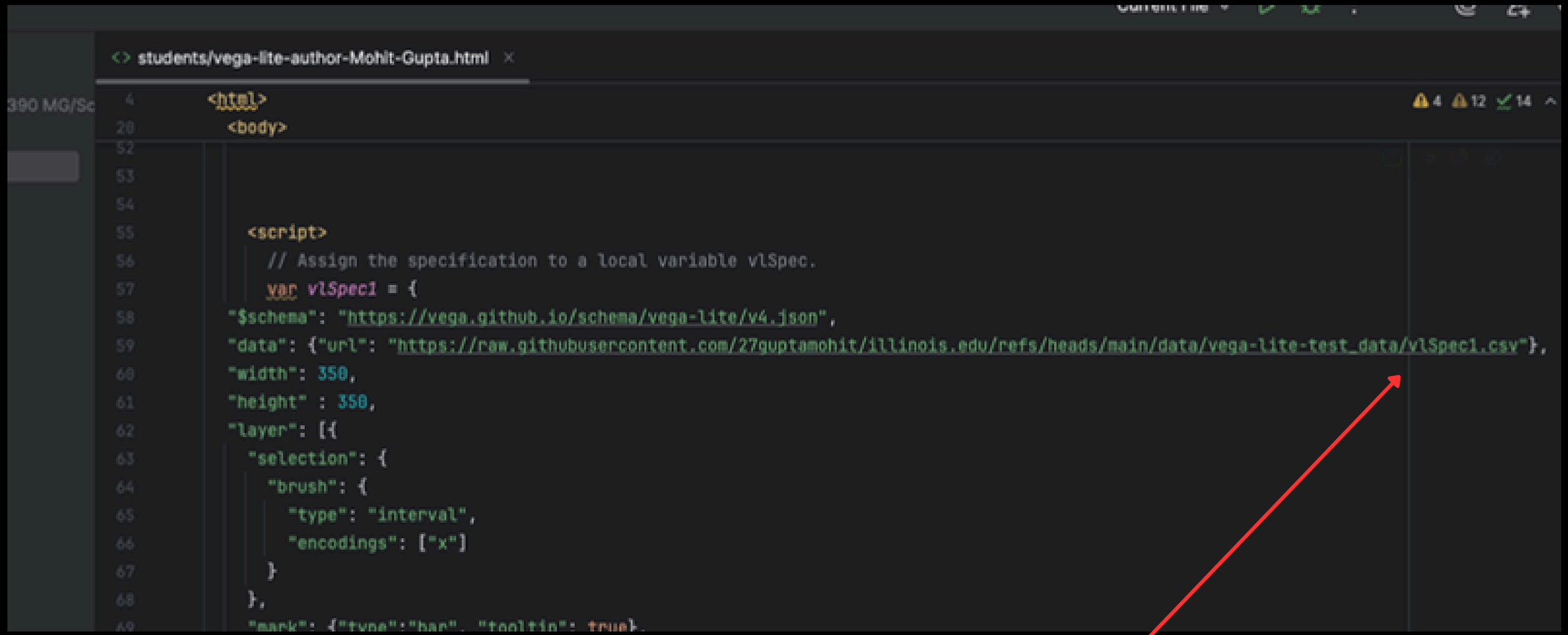
We'll pull data from here.



Step 5:
This API projects the data in the csv format.

- **I'm using this API link's data for generating the visualizations.**

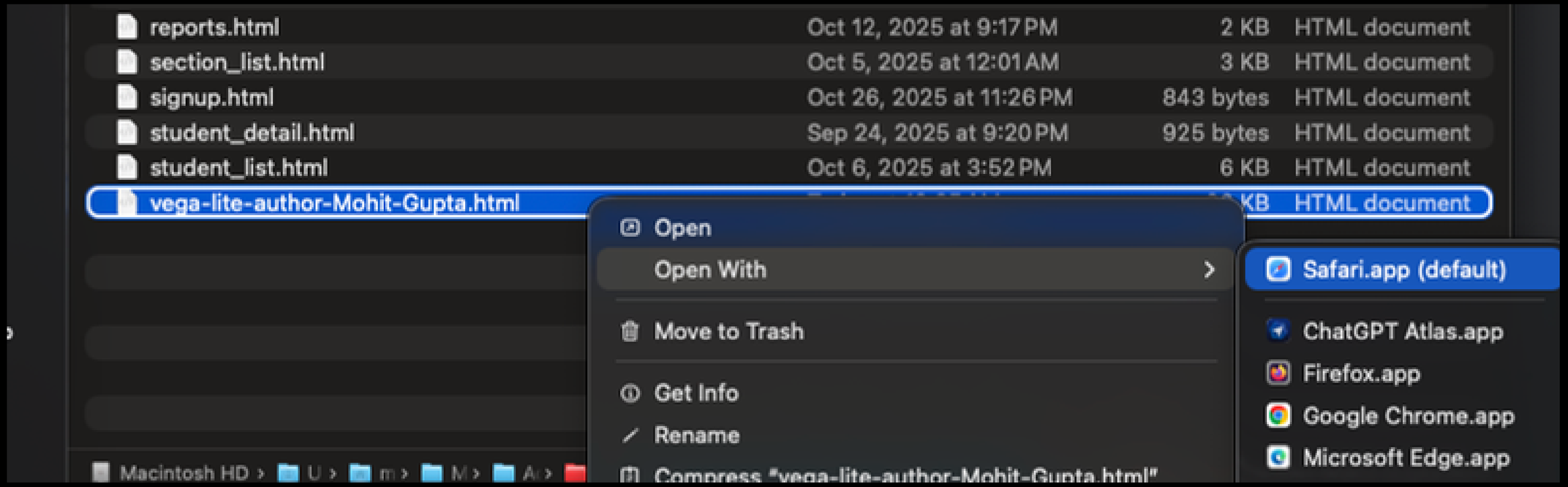
Consider this an example of how our website can project data on an API in csv or json format.



I've pasted the data link for respective codes here in the "data" field.

The data field gives you freedom to plug in data directly or from the API. The data can be csv or json format.

You can also open this specific file in the web-browser without running Djanogo's runserver, and it will work fine!



January 02, 2021

Visualizations 4 to 8(Connected):

Please scrub inside the First Graph. Rest of the graphs shows the details for the selected time frame.

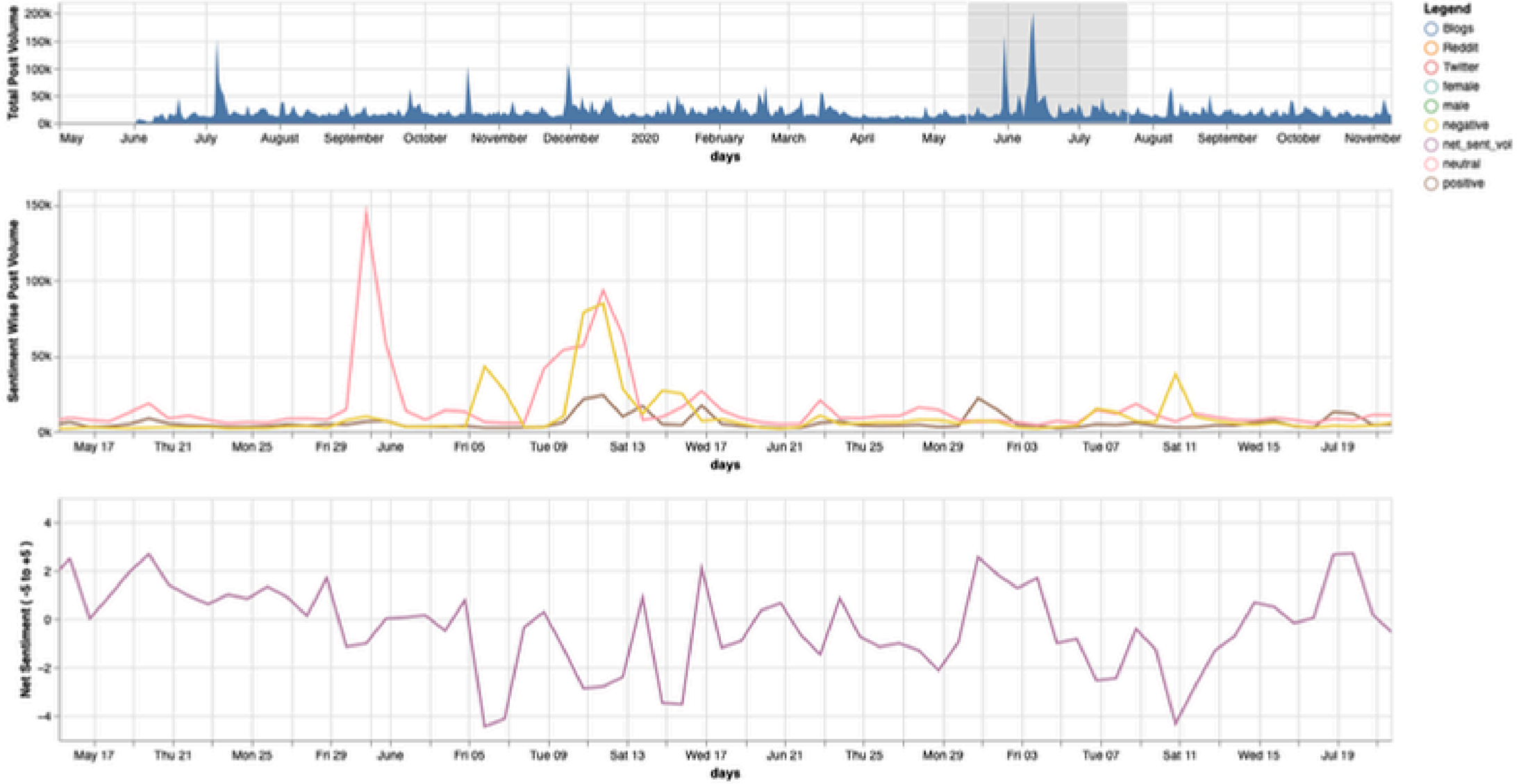
Every component is built and assembled from scratch. Vega-lite gives you power to do so at microscopic level.

January 02, 2021

Visualizations 4 to 8(Connected):

Please scrub inside the First Graph. Rest of the graphs shows the details for the selected time frame.

Visuals Three to Eight

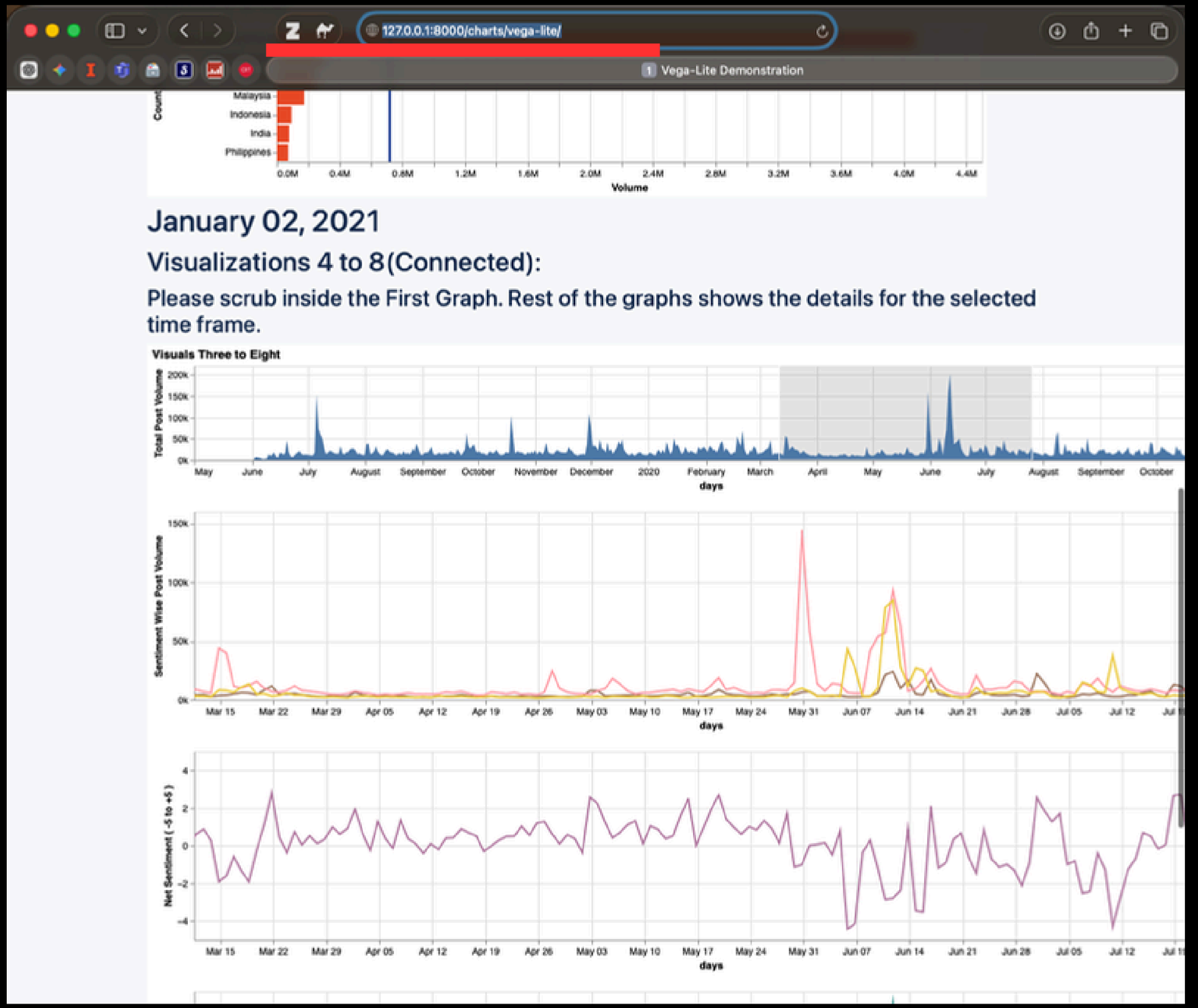


Let’s integrate this into our website through the demo-scripts.

When you'll publish your websites online,
you can project your data privately or publically on the API,
and use that data to create Vega-Lite charts.

Alternatively, you also have Vega-Lite's python library which you can use to develop the charts without the need of an API.

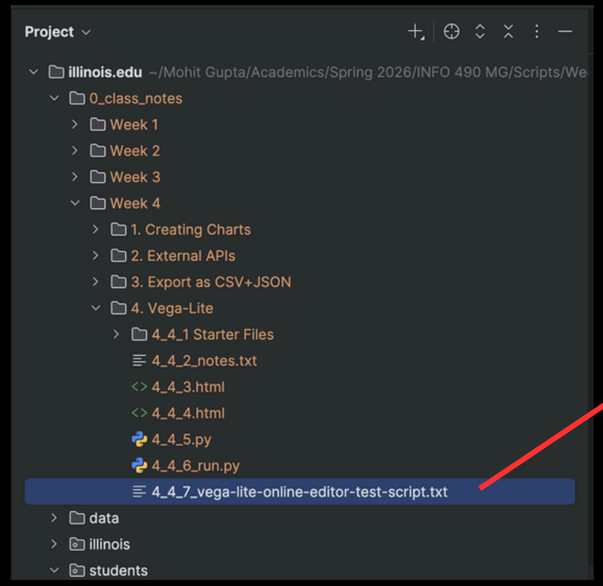
Vega-Lite was to demonstrate the unique ways an API is used by websites all over the world to make its unique uses.

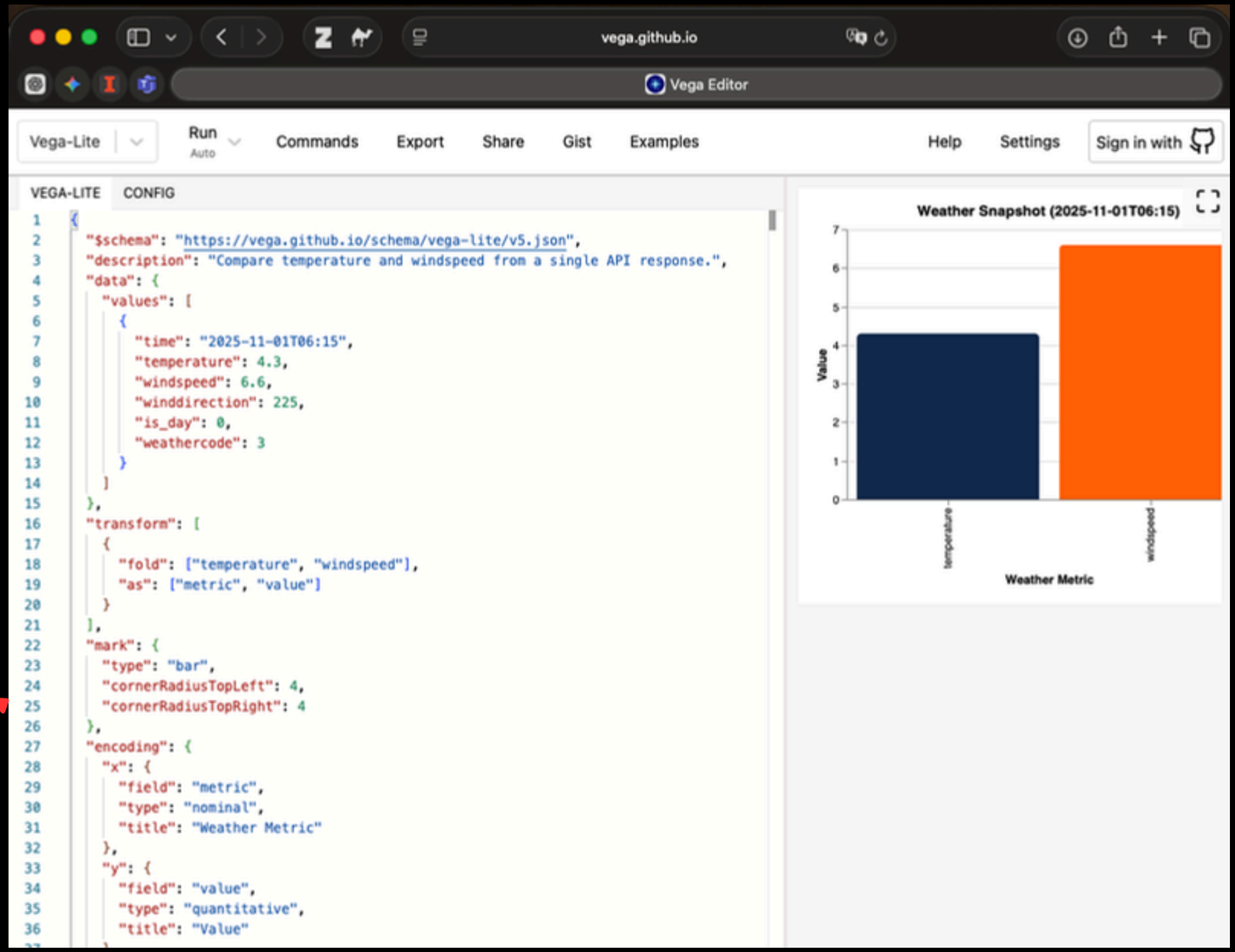

127.0.0.1:8000/charts/vega-lite/
Vega-Lite Demonstration
Malaysia
Indonesia
India
Philippines
Count
0.0M
0.4M
0.8M
1.2M
1.6M
2.0M
2.4M
2.8M
3.2M
3.6M
4.0M
4.4M
Volume
January 02, 2021
Visualizations 4 to 8(Connected):
Please scrub inside the First Graph. Rest of the graphs shows the details for the selected time frame.
Visuals Three to Eight
Total Post Volume
200k
150k
100k
50k
0k
May
June
July
August
September
October
November
December
2020
February
March
April
May
June
July
August
September
October
days
Sentiment Wise Post Volume
150k
100k
50k
0k
Mar 15
Mar 22
Mar 29
Apr 05
Apr 12
Apr 19
Apr 26
May 03
May 10
May 17
May 24
May 31
Jun 07
Jun 14
Jun 21
Jun 28
Jul 05
Jul 12
days
Net Sentiment (-5 to +5)
4
2
0
-2
-4
Mar 15
Mar 22
Mar 29
Apr 05
Apr 12
Apr 19
Apr 26
May 03
May 10
May 17
May 24
May 31
Jun 07
Jun 14
Jun 21
Jun 28
Jul 05
Jul 12
days

Open: https://vega.github.io/editor/

And paste the code in file 15_3.txt in this editor.

Instead of providing the link to the data, I directly pasted it into the editor to try it before using the actual api link.



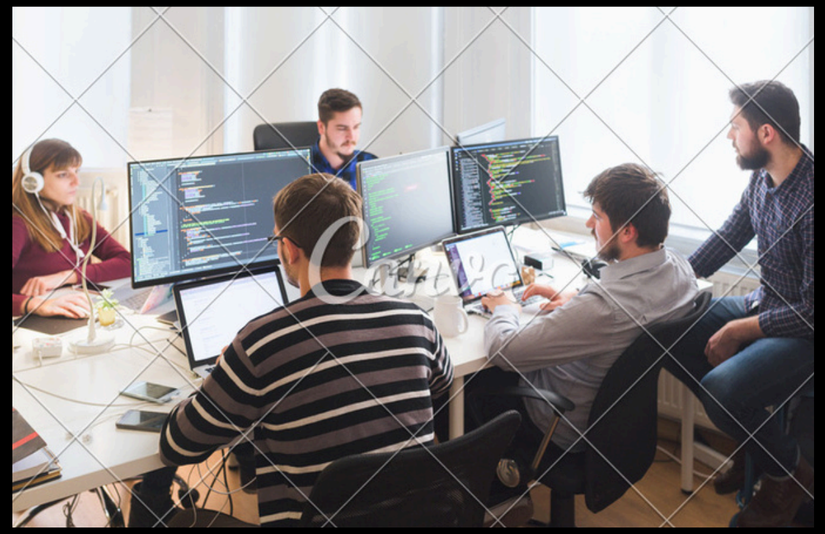


Activity time!

5. User Authentication (Django’s Auth.)

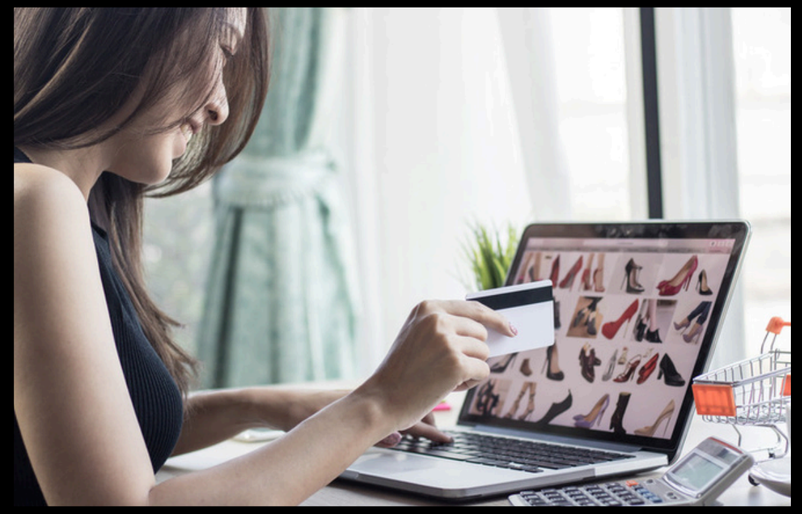
a. Internal Users
b. External Users

Users



- Developers
- Employees

(Current sub-ection)



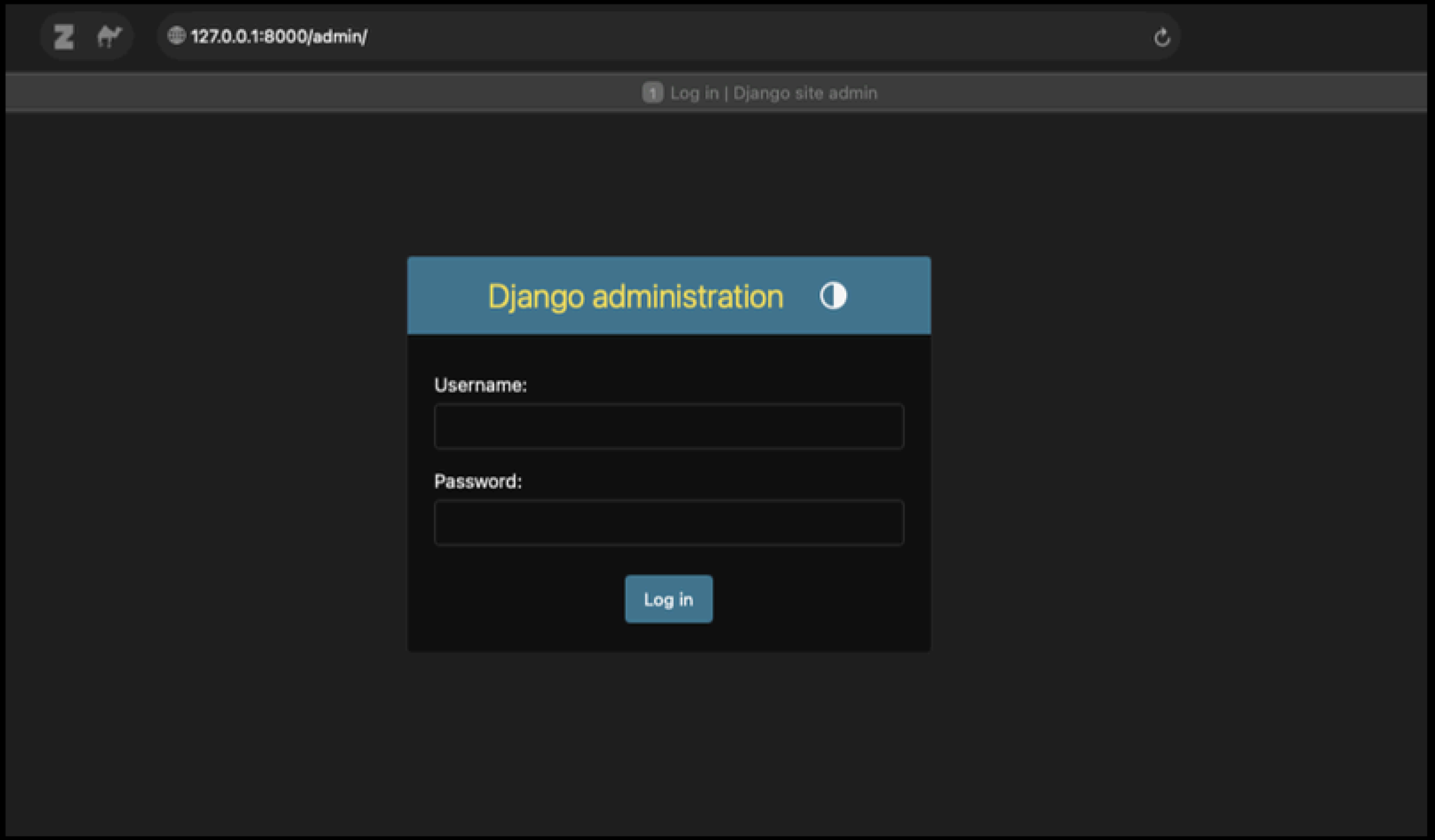
- Non-employees

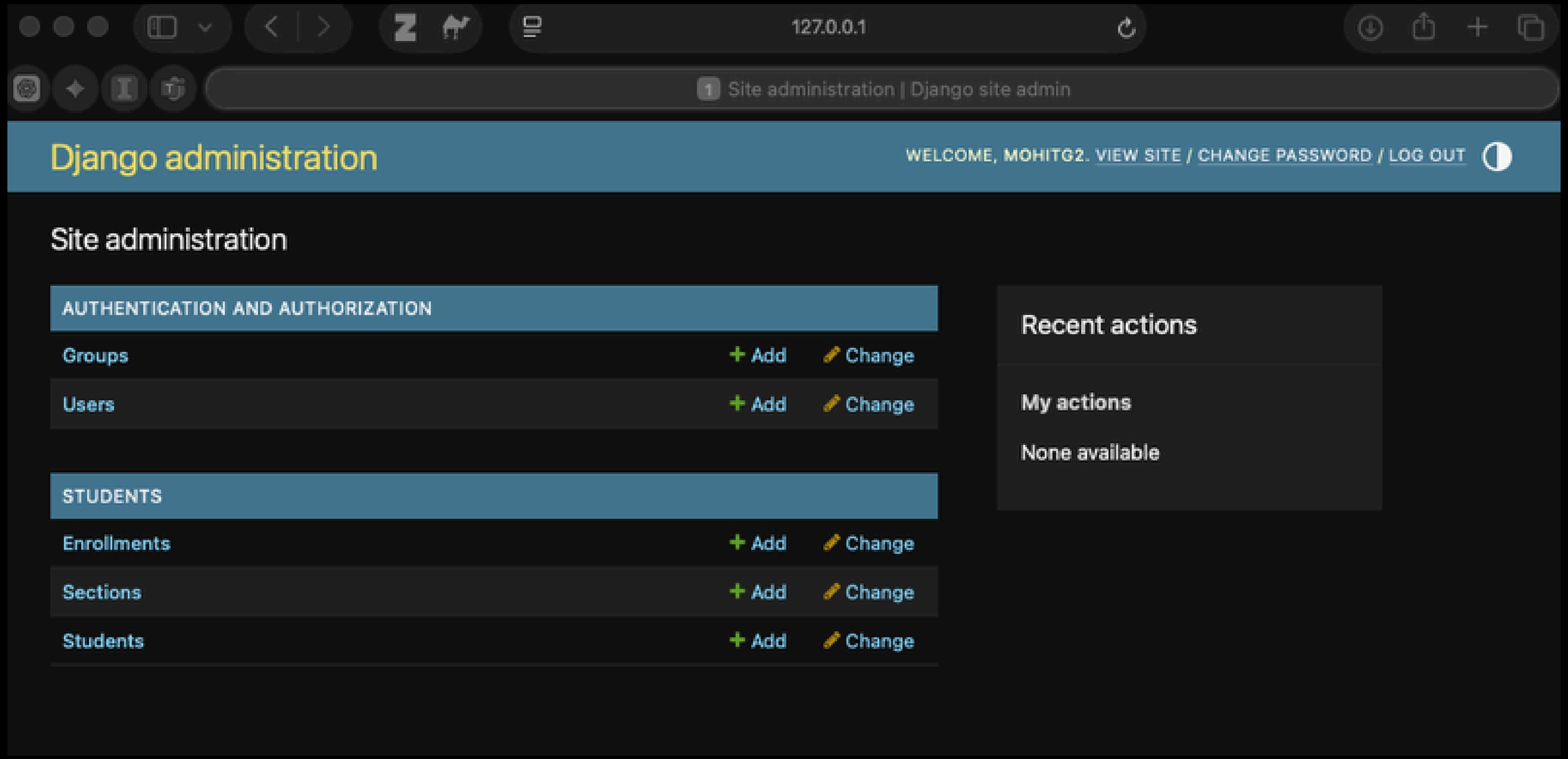
(client sign-up using google OAuth: upcoming sub-section)

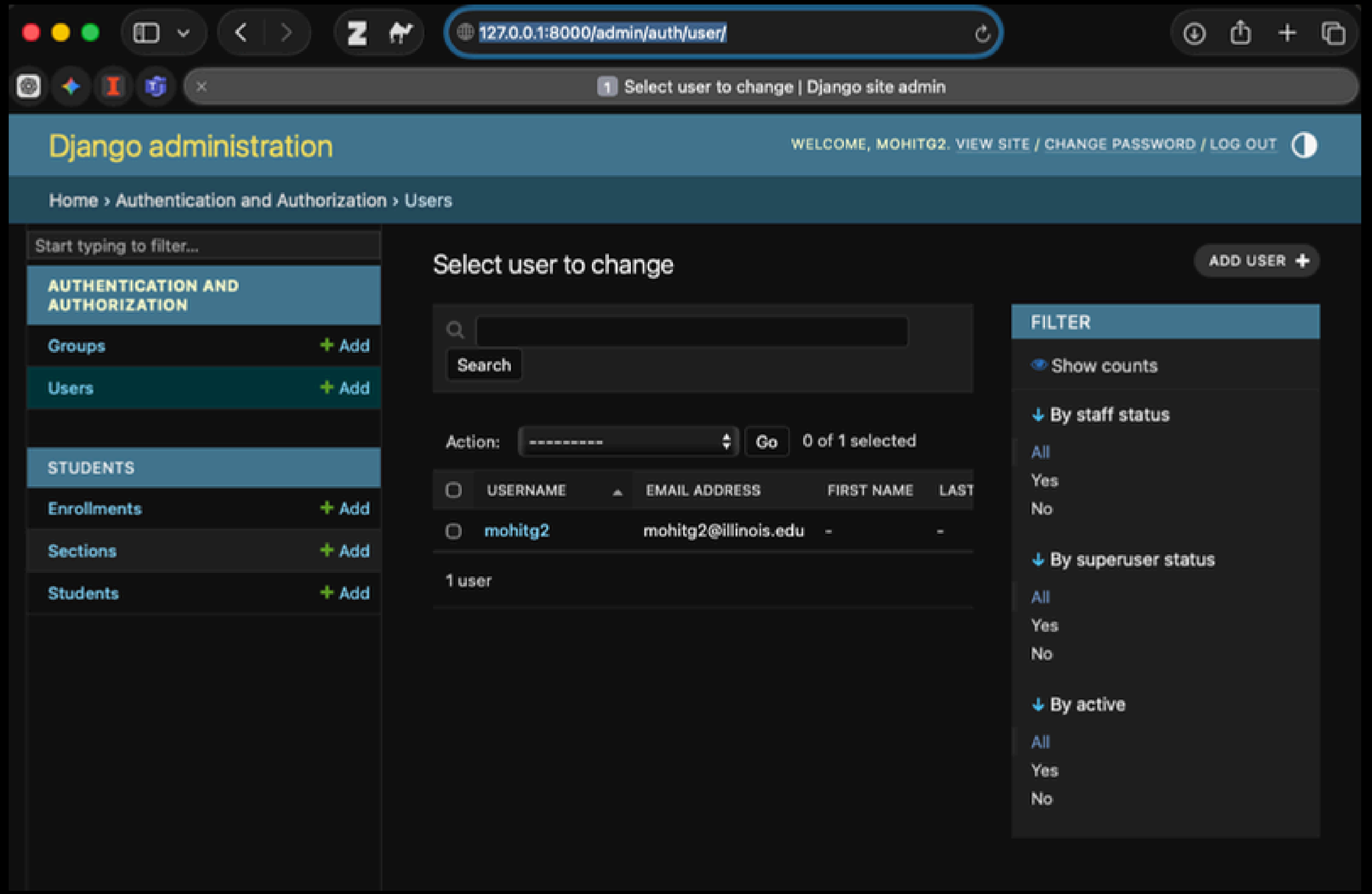
5.a Internal Users

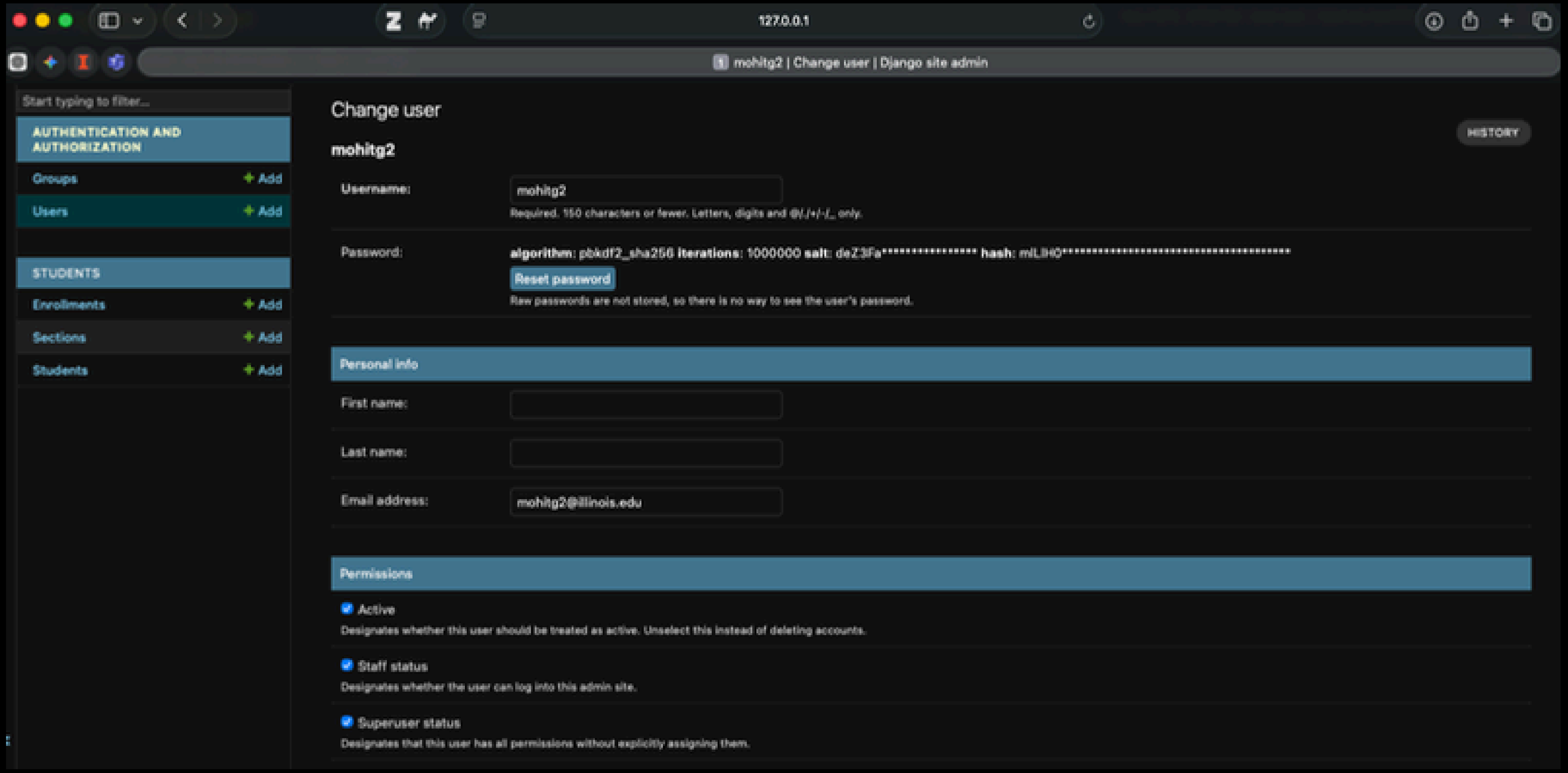
1. The layman approach:

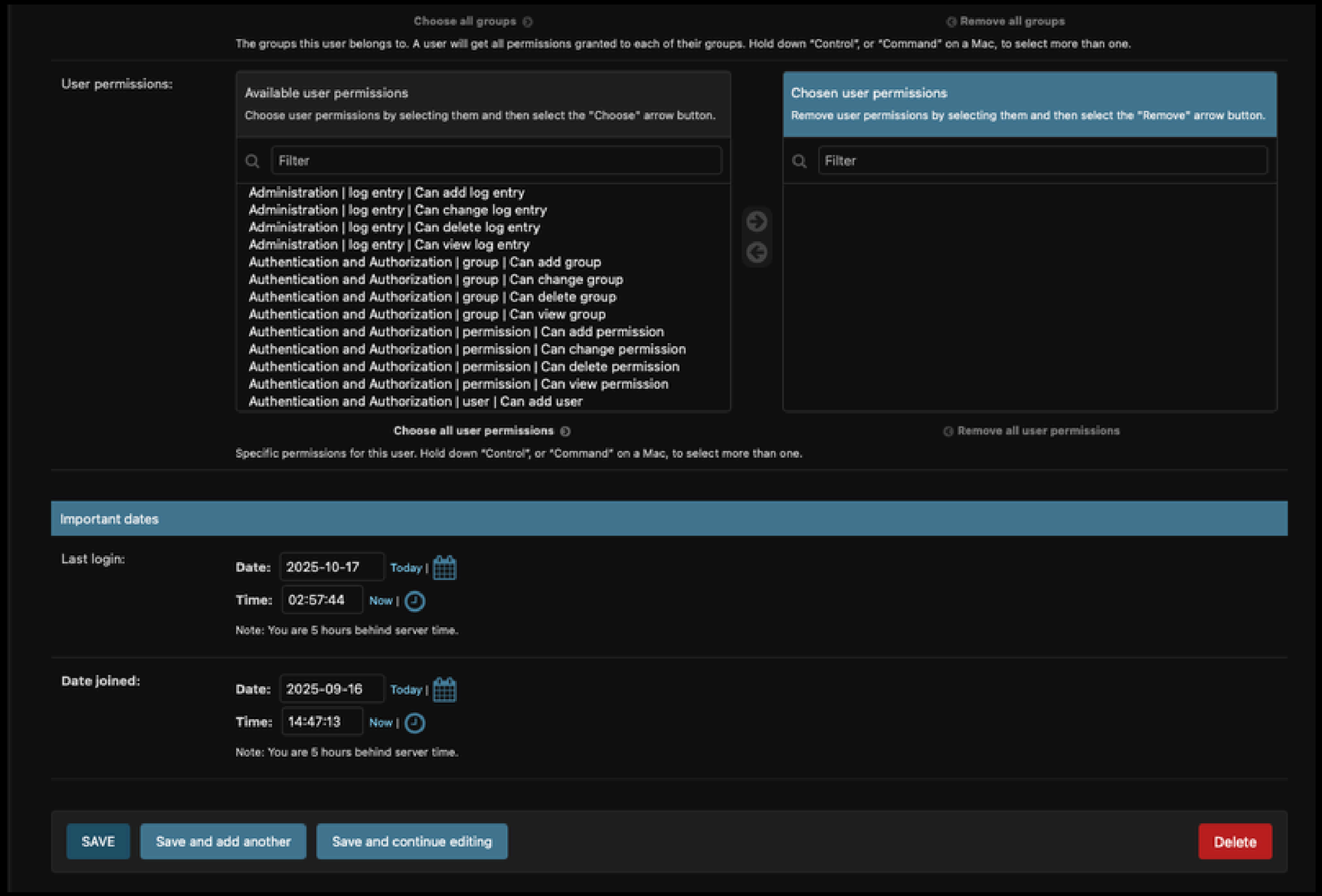
Manually register the internal users on the website using admin panel


127.0.0.1:8000/admin/
Log in | Django site admin
Django administration
Username:
Password:
Log in


127.0.0.1
Site administration | Django site admin
Django administration
WELCOME, MOHITG2. VIEW SITE / CHANGE PASSWORD / LOG OUT
Site administration
AUTHENTICATION AND AUTHORIZATION
Groups + Add Change
Users + Add Change
STUDENTS
Enrollments + Add Change
Sections + Add Change
Students + Add Change
Recent actions
My actions
None available


127.0.0.1:8000/admin/auth/user/
Select user to change | Django site admin
Django administration
WELCOME, MOHITG2. VIEW SITE / CHANGE PASSWORD / LOG OUT
Home › Authentication and Authorization › Users
Start typing to filter...
AUTHENTICATION AND AUTHORIZATION
Groups + Add
Users + Add
STUDENTS
Enrollments + Add
Sections + Add
Students + Add
Select user to change
ADD USER +
Search
Action: --------- Go 0 of 1 selected
USERNAME
EMAIL ADDRESS
FIRST NAME
LAST
mohitg2
mohitg2@illinois.edu
-
-
1 user
FILTER
Show counts
By staff status
All
Yes
No
By superuser status
All
Yes
No
By active
All
Yes
No


127.0.0.1
mohitg2 | Change user | Django site admin
Start typing to filter...
AUTHENTICATION AND AUTHORIZATION
Groups + Add
Users + Add
STUDENTS
Enrollments + Add
Sections + Add
Students + Add
Change user
HISTORY
mohitg2
Username:
mohitg2
Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.
Password:
algorithm: pbkdf2_sha256 iterations: 1000000 salt: deZ3Fa********** hash: mlL8kG**********
Reset password
Raw passwords are not stored, so there is no way to see the user's password.
Personal info
First name:
Last name:
Email address:
mohitg2@illinois.edu
Permissions
Active
Designates whether this user should be treated as active. Unselect this instead of deleting accounts.
Staff status
Designates whether the user can log into this admin site.
Superuser status
Designates that this user has all permissions without explicitly assigning them.


Choose all groups
Remove all groups
The groups this user belongs to. A user will get all permissions granted to each of their groups. Hold down "Control", or "Command" on a Mac, to select more than one.
User permissions:
Available user permissions
Choose user permissions by selecting them and then select the "Choose" arrow button.
Filter
Administration | log entry | Can add log entry
Administration | log entry | Can change log entry
Administration | log entry | Can delete log entry
Administration | log entry | Can view log entry
Authentication and Authorization | group | Can add group
Authentication and Authorization | group | Can change group
Authentication and Authorization | group | Can delete group
Authentication and Authorization | group | Can view group
Authentication and Authorization | permission | Can add permission
Authentication and Authorization | permission | Can change permission
Authentication and Authorization | permission | Can delete permission
Authentication and Authorization | permission | Can view permission
Authentication and Authorization | user | Can add user
Chosen user permissions
Remove user permissions by selecting them and then select the "Remove" arrow button.
Filter
Choose all user permissions
Remove all user permissions
Specific permissions for this user. Hold down "Control", or "Command" on a Mac, to select more than one.
Important dates
Last login:
Date: 2025-10-17 Today |
Time: 02:57:44 Now |
Note: You are 5 hours behind server time.
Date joined:
Date: 2025-09-16 Today |
Time: 14:47:13 Now |
Note: You are 5 hours behind server time.
SAVE
Save and add another
Save and continue editing
Delete

2. Better approach

- Create a login page.
- Let internal users enter the website using it.

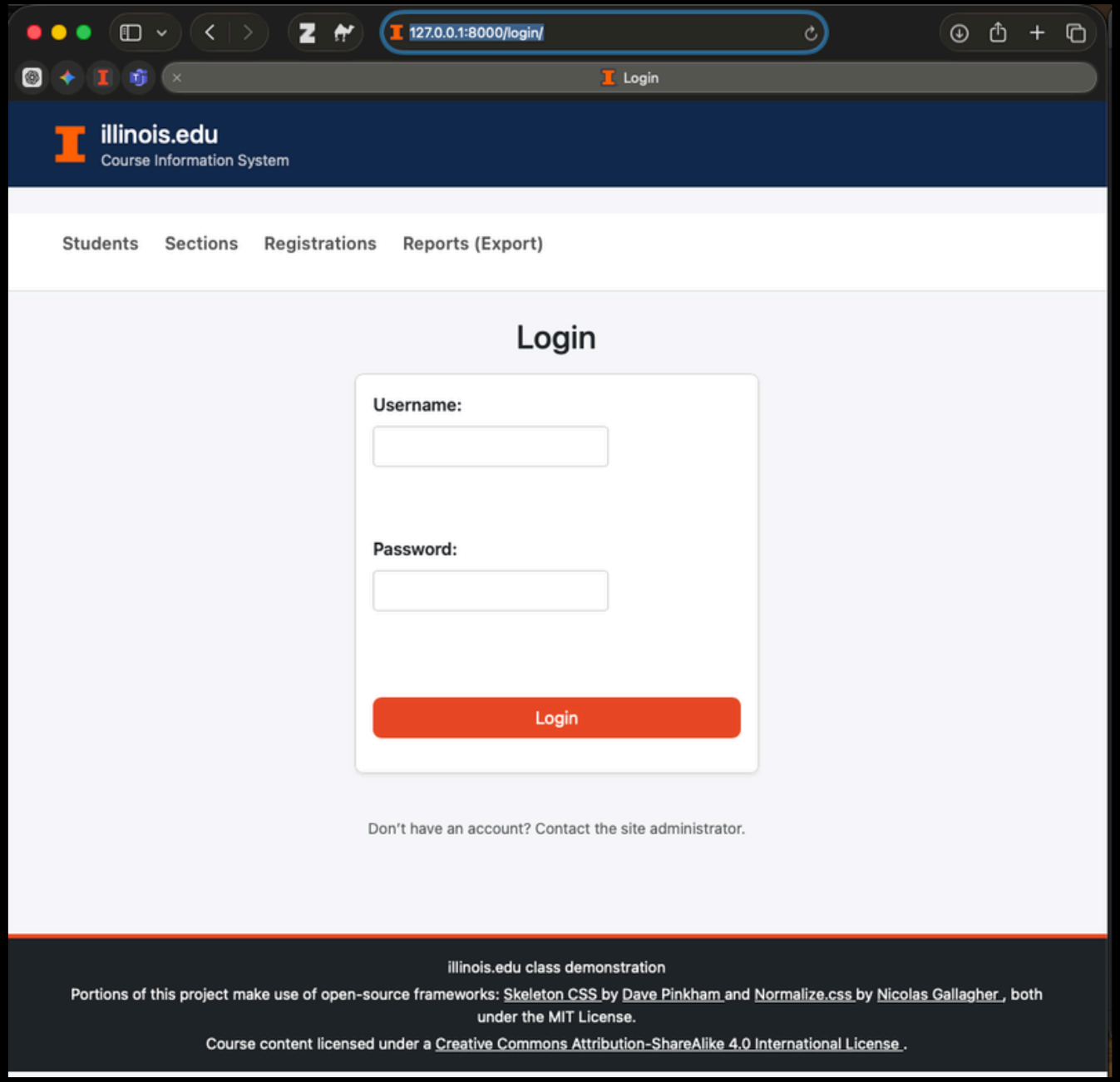
Let’s begin script integration.

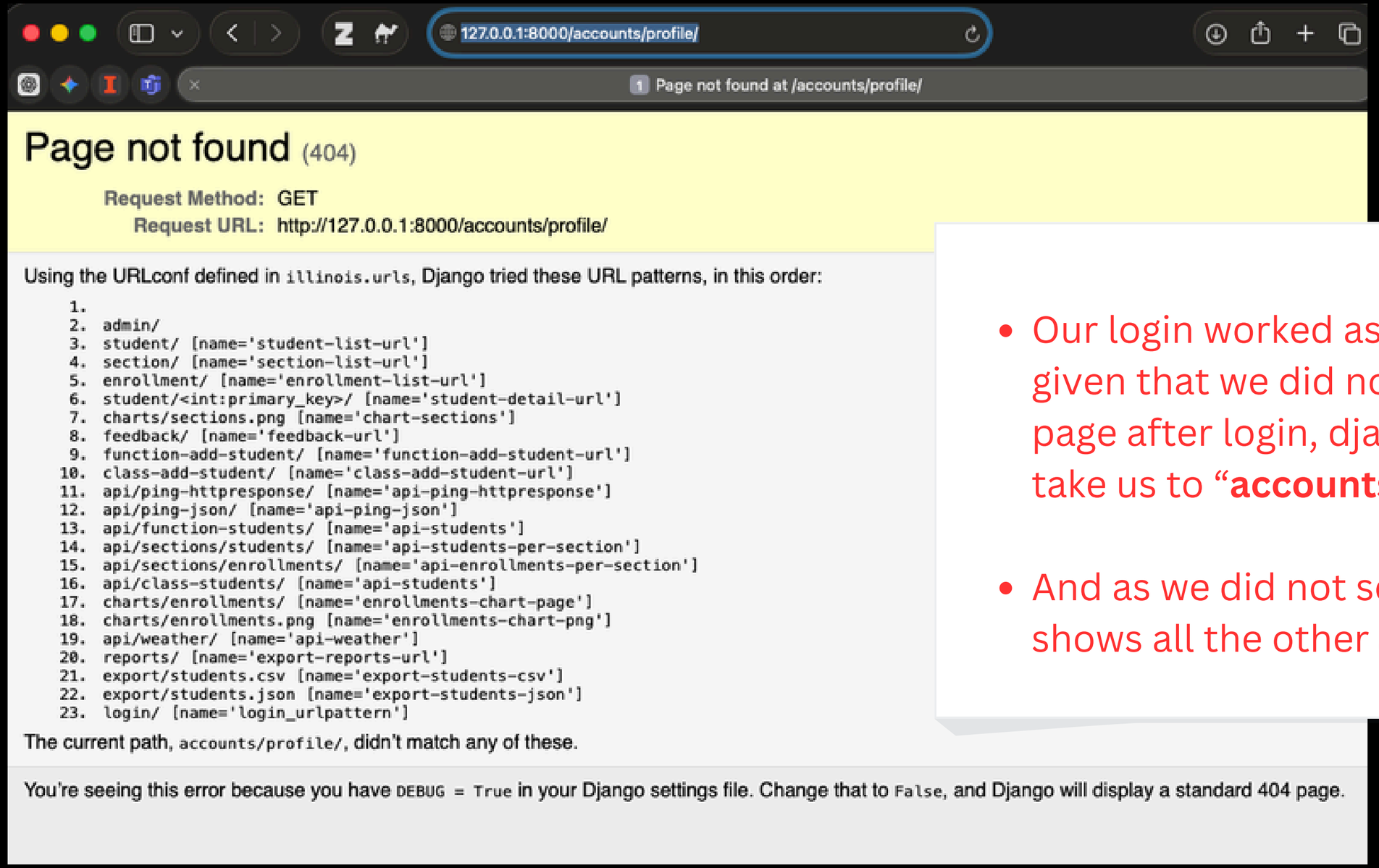
```
23    # new addition/changes: LoginView
24    path( route: 'login/',
25 <>      LoginView.as_view(template_name='students/login.html'),
26        name='login_urlpattern'),
27
28 ]
29
```

urls.py | file1_1.py | student_list.html | login.html

```
1  <!-- Paste code here: templates/students/login.html -->
2  <!-- Writing new code for displaying the aggregated data. -->
3
4  {% extends "students/base.html" %}
5
6  {% block title %}
7      Login
8  {% endblock %}
9
10 {% block content %}
11
12     <h2>Login</h2>
13     <form method="POST">
14     {% csrf_token %}
15     {{ form.as_p }}
16     <button type="submit">Login</button>
17     </form>
18
19 {% endblock %}
```

Output.

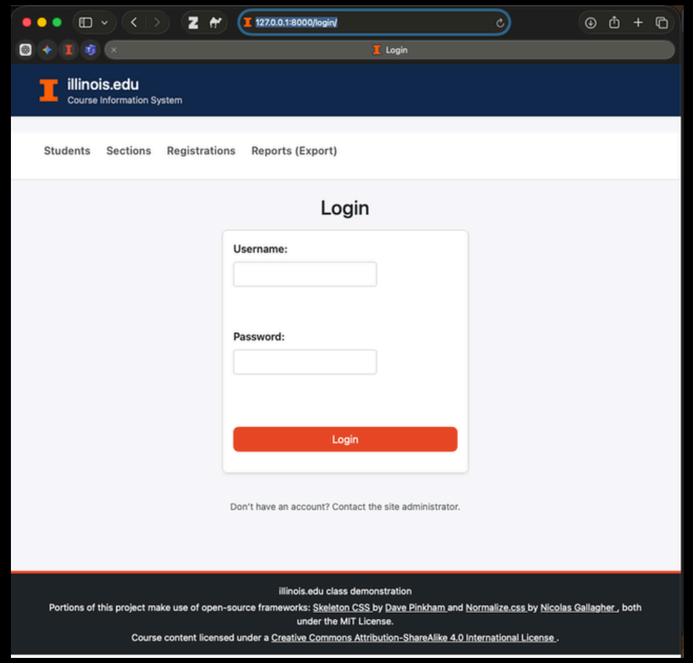

127.0.0.1:8000/login/
Login
illinois.edu
Course Information System
Students
Sections
Registrations
Reports (Export)
Login
Username:
Password:
Login
Don't have an account? Contact the site administrator.
illinois.edu class demonstration
Portions of this project make use of open-source frameworks: Skeleton CSS by Dave Pinkham and Normalize.css by Nicolas Gallagher, both under the MIT License.
Course content licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.

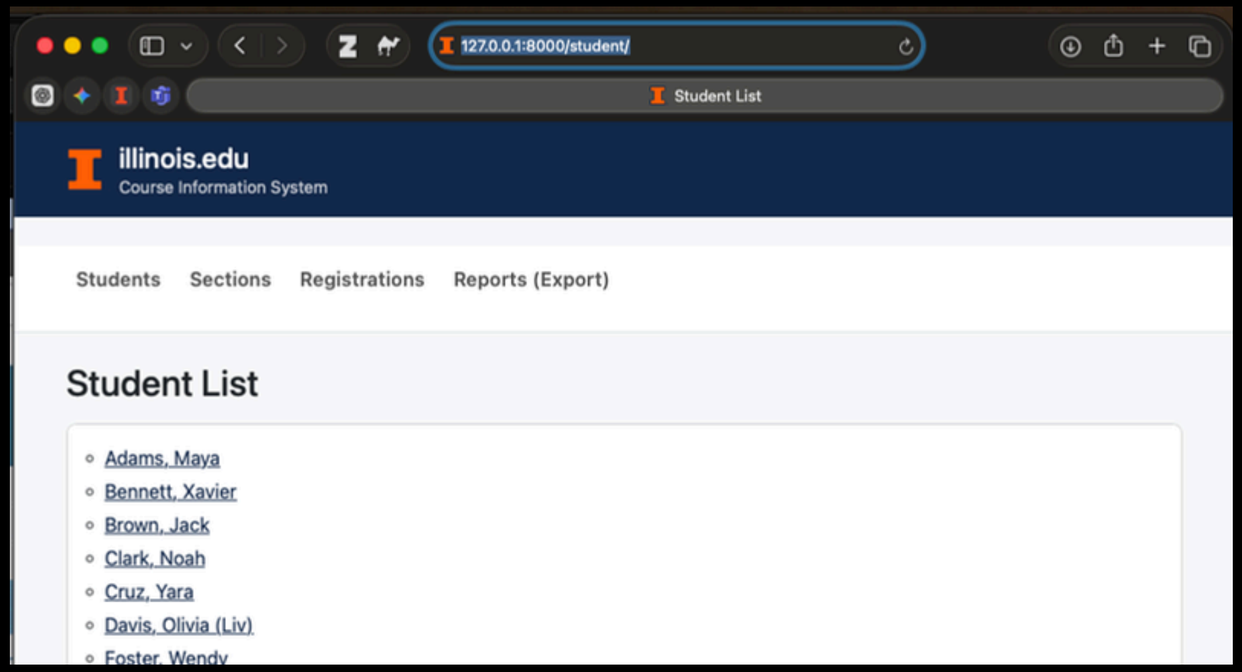


- Our login worked as expected but given that we did not set any home page after login, django is trying to take us to “**accounts/profile**”
- And as we did not set it up yet, it shows all the other available links.

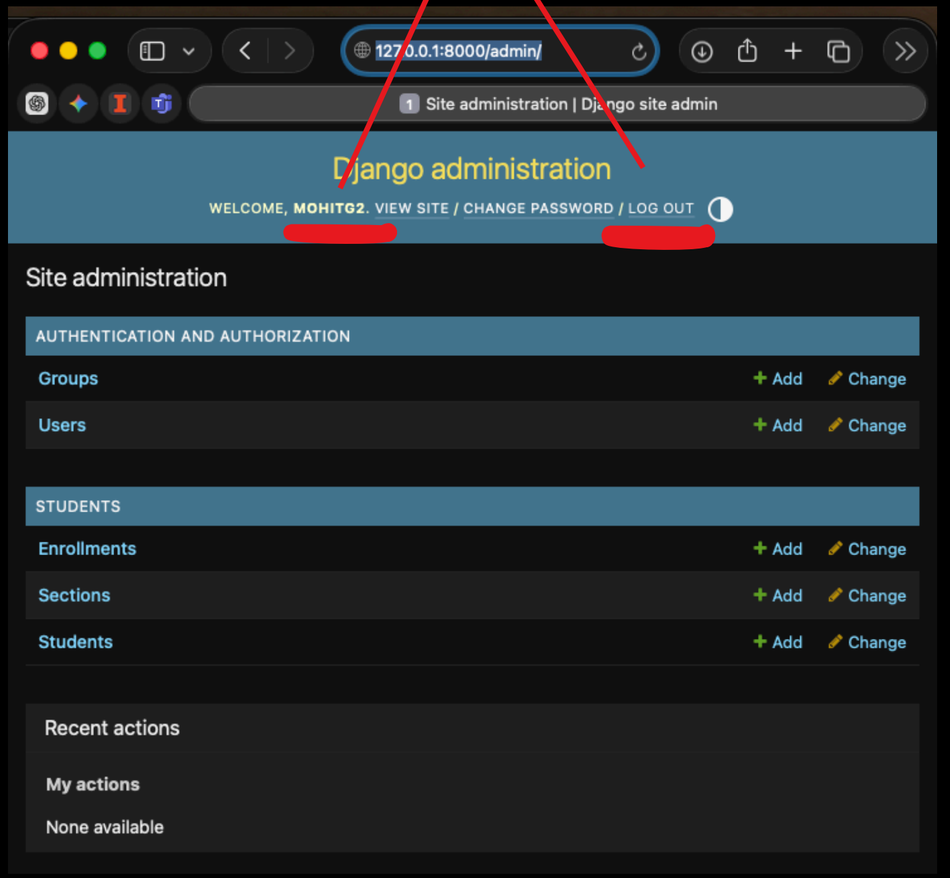
Let’s change it in settings.py

Currently, to login, you need to use this link.





And to logout, you've to go to admin panel to log-out. Let's change this



So, let’s build a URL path for Logout feature.

illinois/urls.py

```
26
27      path( route: 'logout/',
28            LogoutView.as_view(),
29            name='logout_urlpattern'),
30  ]
31
```

settings.py

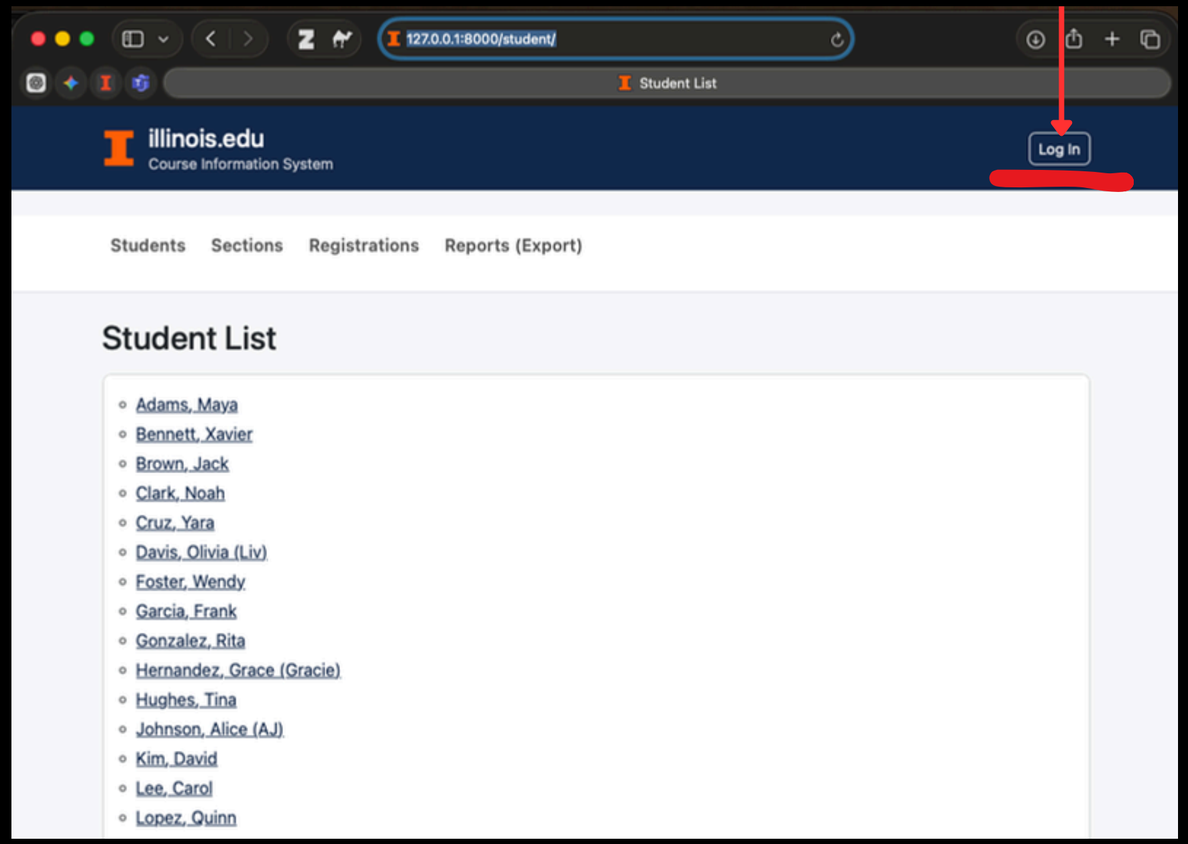
settings.py

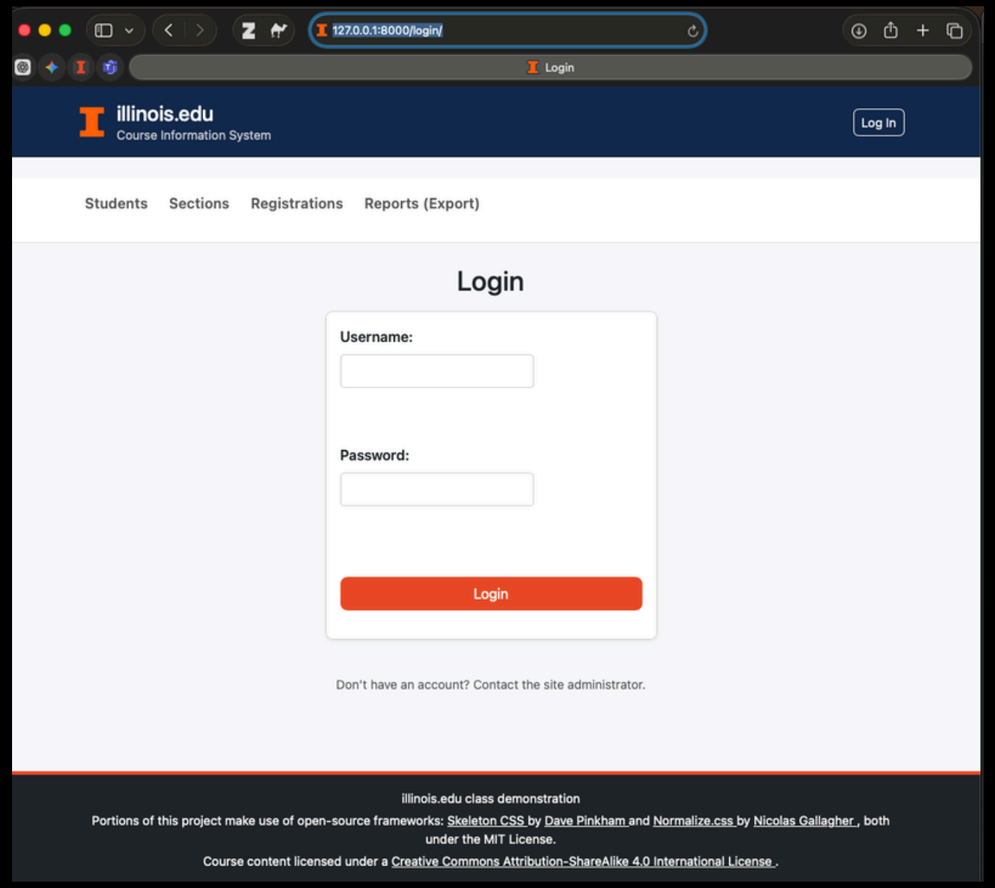
```
267
268  # New addition/changes:
269  # This will set where the user will go after logging out.
270  # I'm entering the "URL NAME PATTERN" for the 'login/' url here = 'login_urlpattern'
271  # You can locate it in the students/urls.py
272  LOGOUT_REDIRECT_URL = 'login_urlpattern'
273
274
275
```

base.html

```
45      <!-- Right: Login / Logout links -->
46      <ul class="nav ms-auto">
47        {% if user.is_authenticated %}
48
49          <form action="{% url 'logout_urlpattern' %}" method="post" class="ms-auto">
50
51            <!--IMPORTANT: If you do not include csrf token tag here,
52            your logout will not work as intended. -->
53            {% csrf_token %}
54
55            <button class="btn btn-sm btn-outline-light" type="submit">
56                Log Out, {{ user.get_username }}
57            </button>
58
59          </form>
60        {% else %}
61
62          <a class="btn btn-sm btn-outline-light ms-auto"
63            href="{% url 'login_urlpattern' %}">Log In</a>
64
65        {% endif %}
66      </ul>
```

new
login/logout
button





But we still need to make sure that the data through navigation pane is only visible after the login.

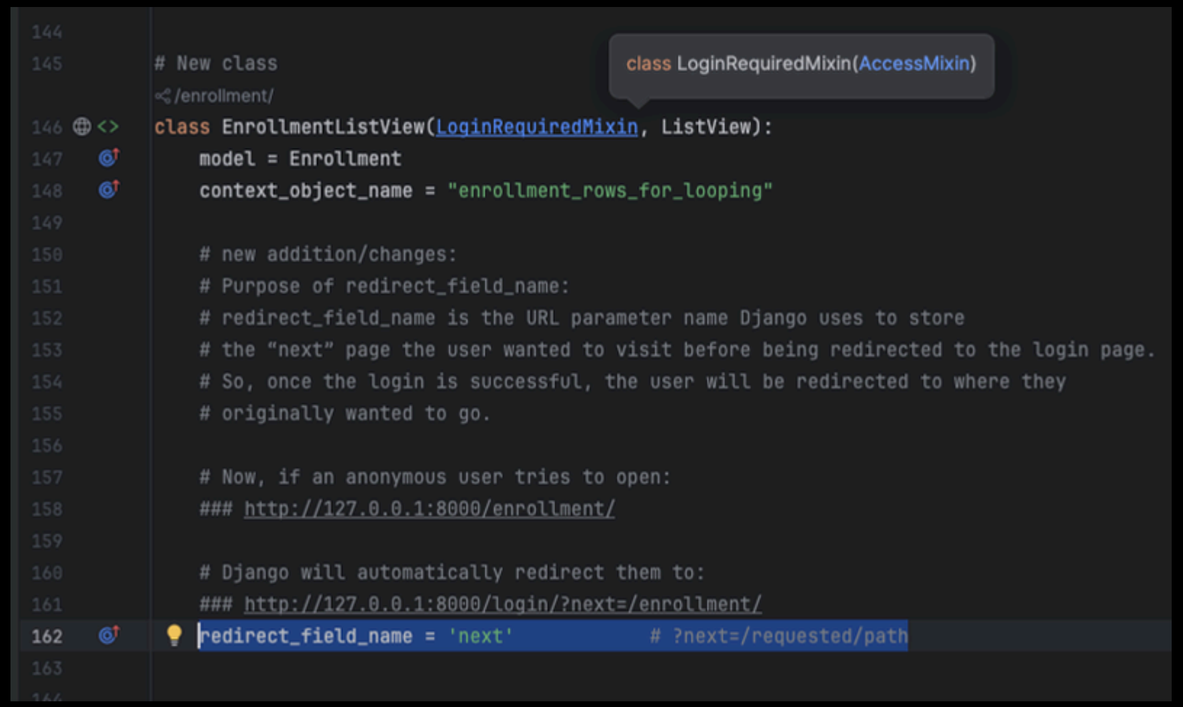
This sets the “after Login-Logout” behavior

```
265
266  # New addition/changes: (already done in previous steps)
267  # This will set where the user will go after logging out.
268  # I'm entering the "URL NAME PATTERN" for the 'login/' url here = 'login_urlpattern'
269  # You can locate it in the students/urls.py
270
271  # This will also set a universal/global behavior that where to take user
272  # after they login/logout
273  LOGIN_REDIRECT_URL = 'student-list-url'
274  LOGOUT_REDIRECT_URL = 'login_urlpattern'
275
```

views.py

Added:

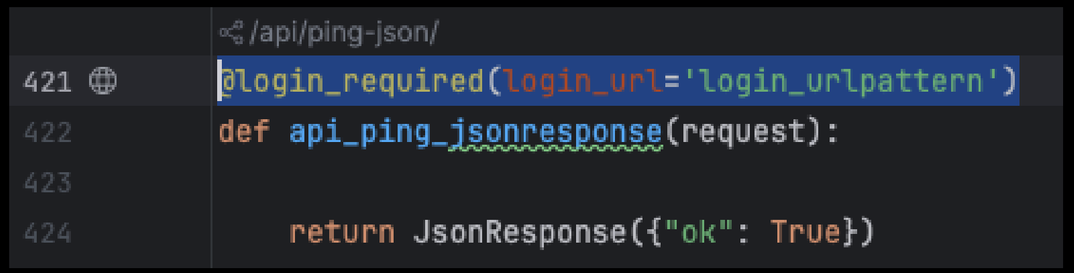
- **LoginRequiredMixin : For classes**



views.py

Added:

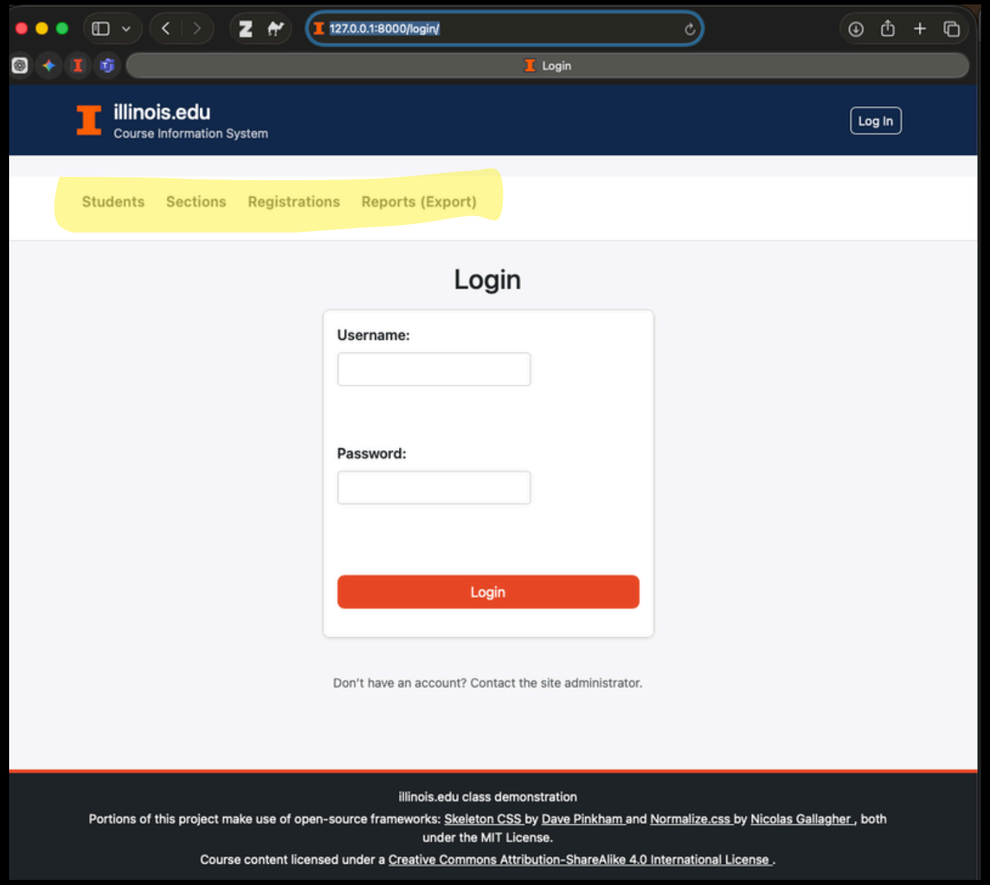
- **redirect_field_name : For functions**



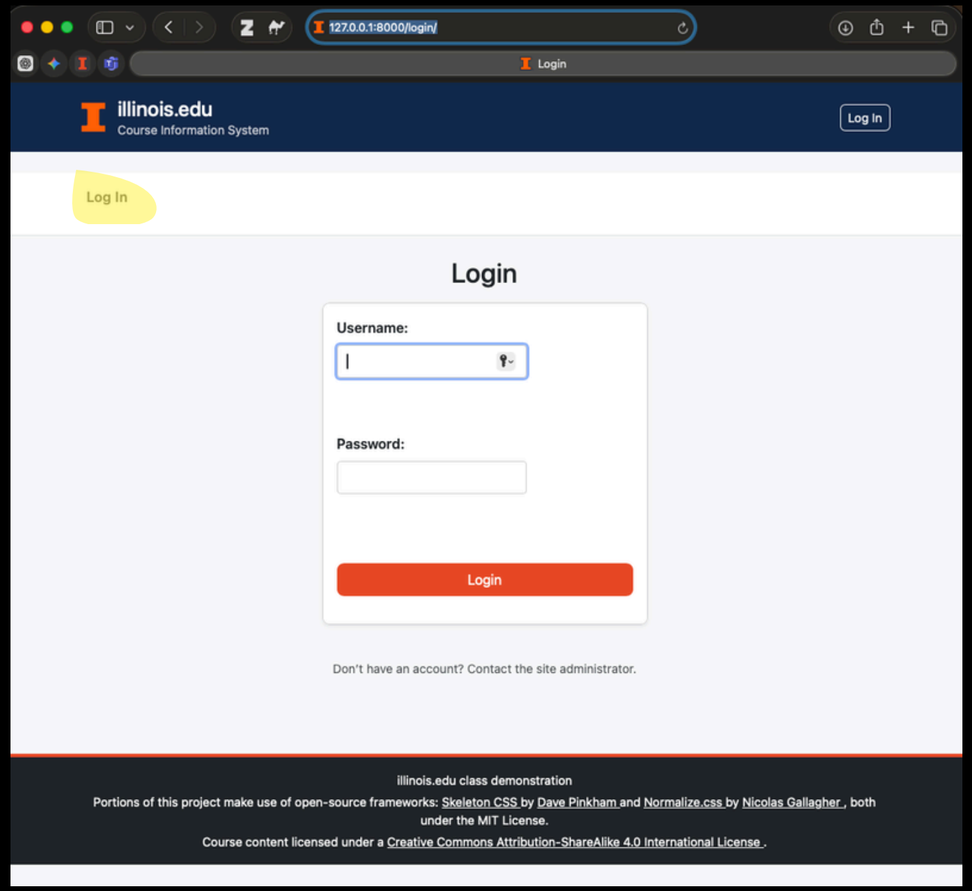
base.html

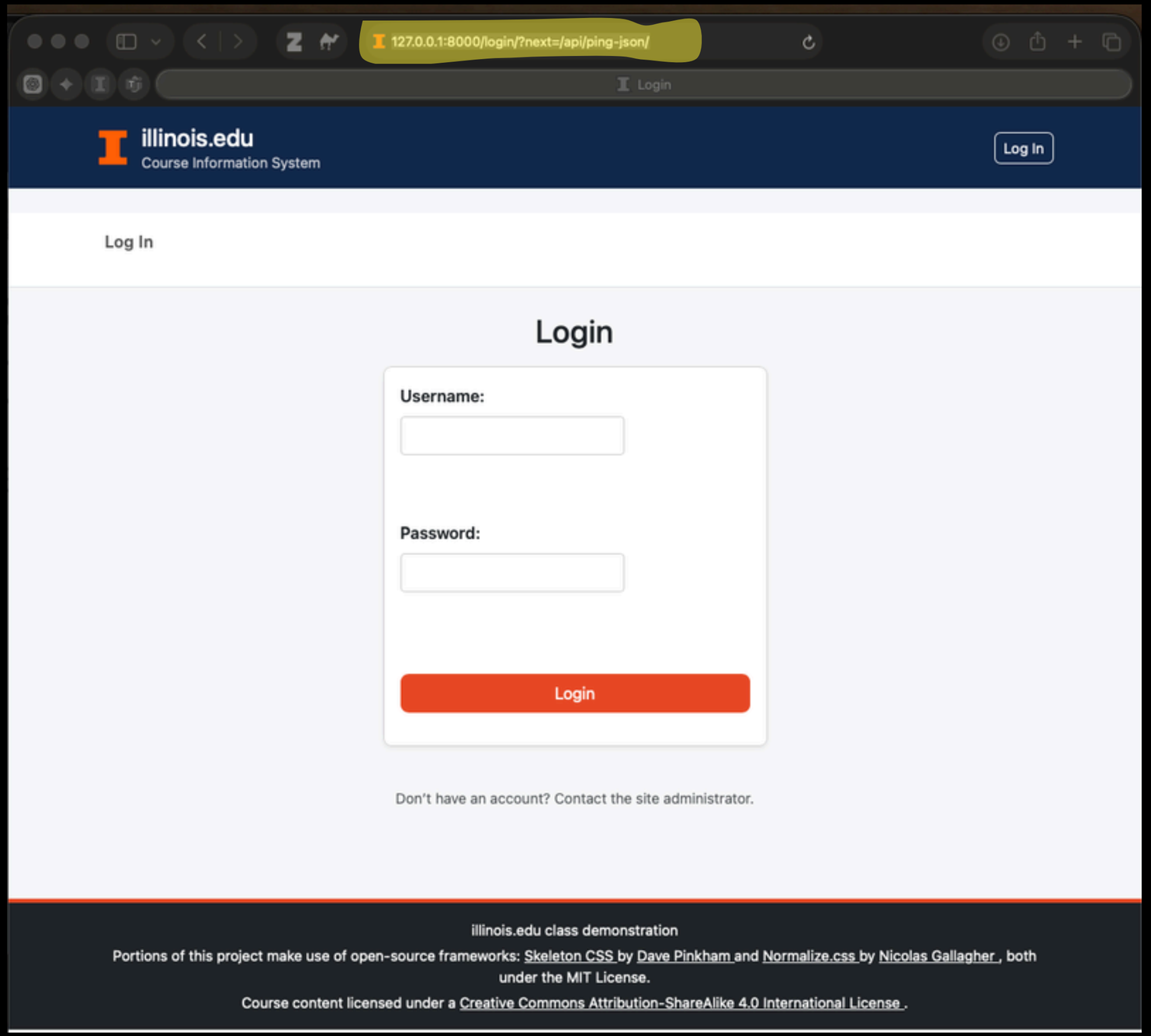
```
<!-- Navigation -->
<!-- New addition/changes: user login required to view data -->
<nav class="navbar navbar-expand-lg navbar-light bg-white border-bottom mb-4" aria-label="Primary">
  <div class="container">

    <ul class="navbar-nav gap-2">
    {% if user.is_authenticated %}
      <li class="nav-item"><a class="nav-link" href="{% url 'student-list-url' %}">Students</a></li>
      <li class="nav-item"><a class="nav-link" href="{% url 'section-list-url' %}">Sections</a></li>
      <li class="nav-item"><a class="nav-link" href="{% url 'enrollment-list-url' %}">Registrations</a></li>
      <li class="nav-item"><a class="nav-link" href="{% url 'export-reports-url' %}">Reports (Export)</a></li>
    {% else %}
      <li class="nav-item"><a class="nav-link" href="{% url 'login_urlpattern' %}">Log In</a></li>
    {% endif %}
  </ul>
```

previously when no user authentication required

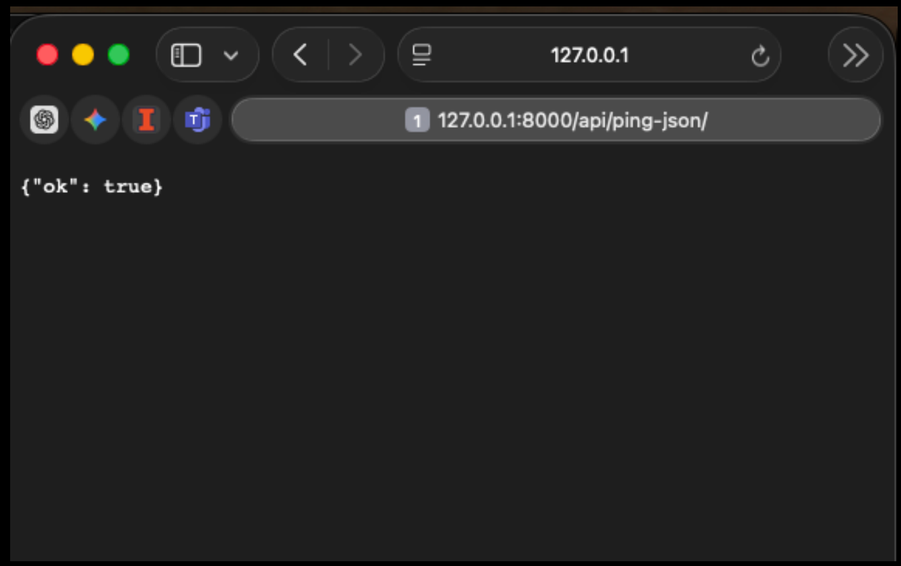


Now, when user authentication is required



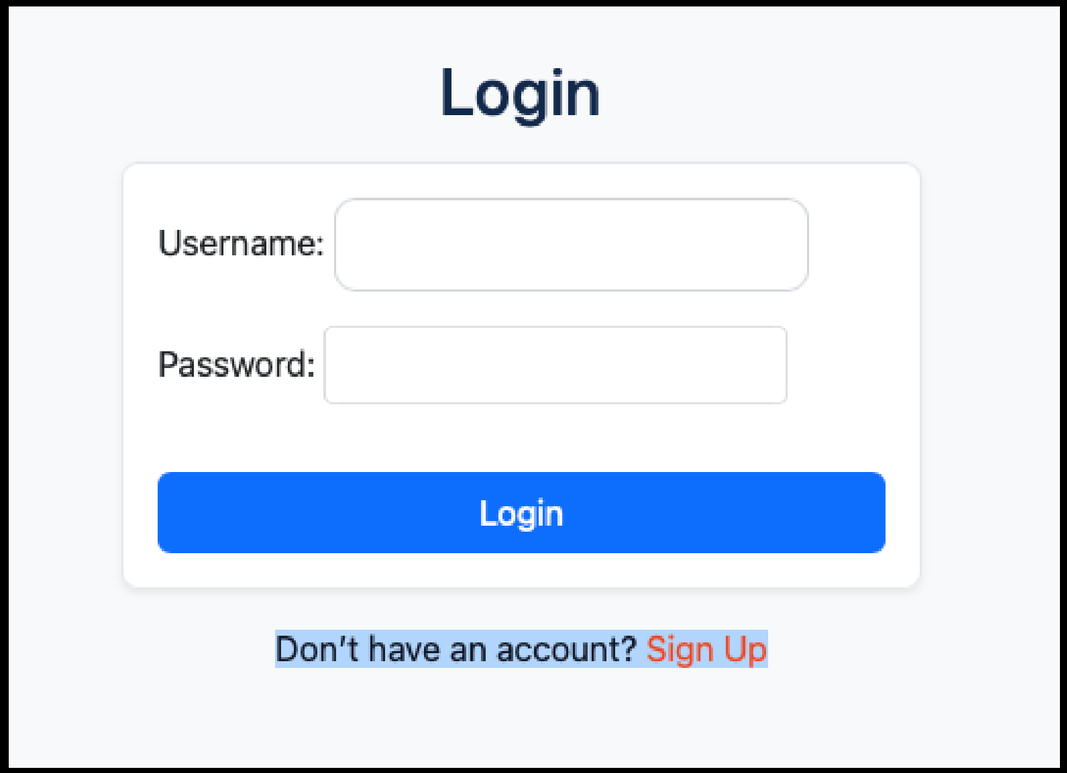


Now after the user authentication, you can access the api data



5.b External Users

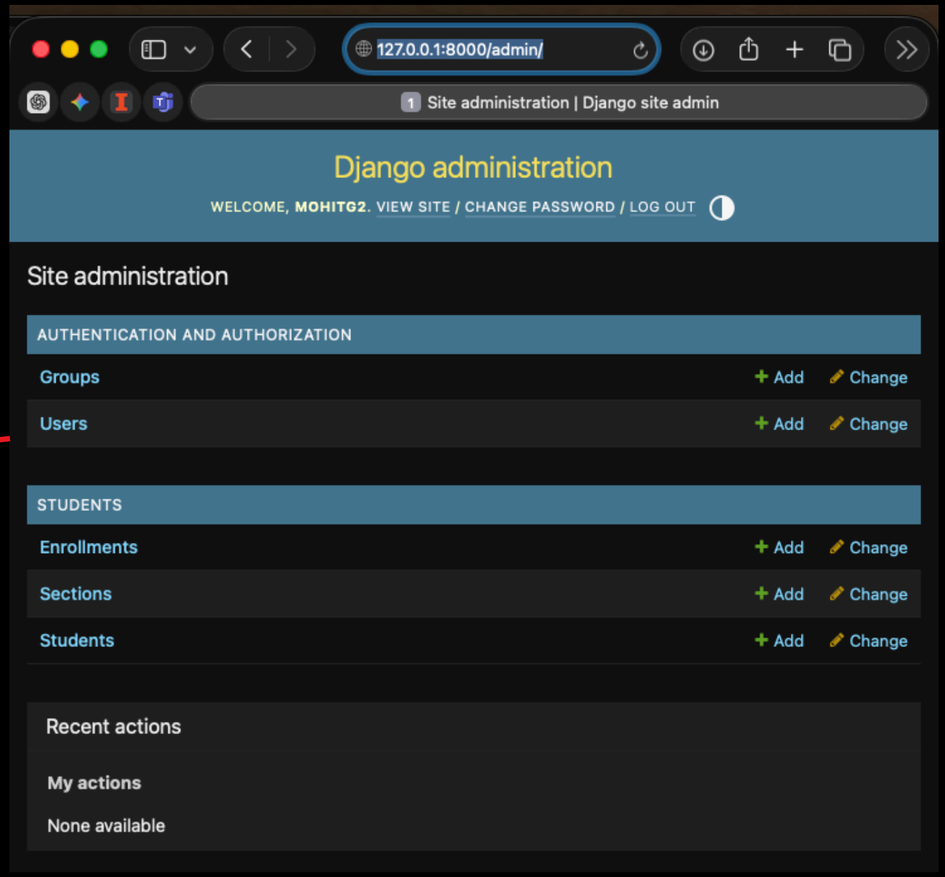
Final Output



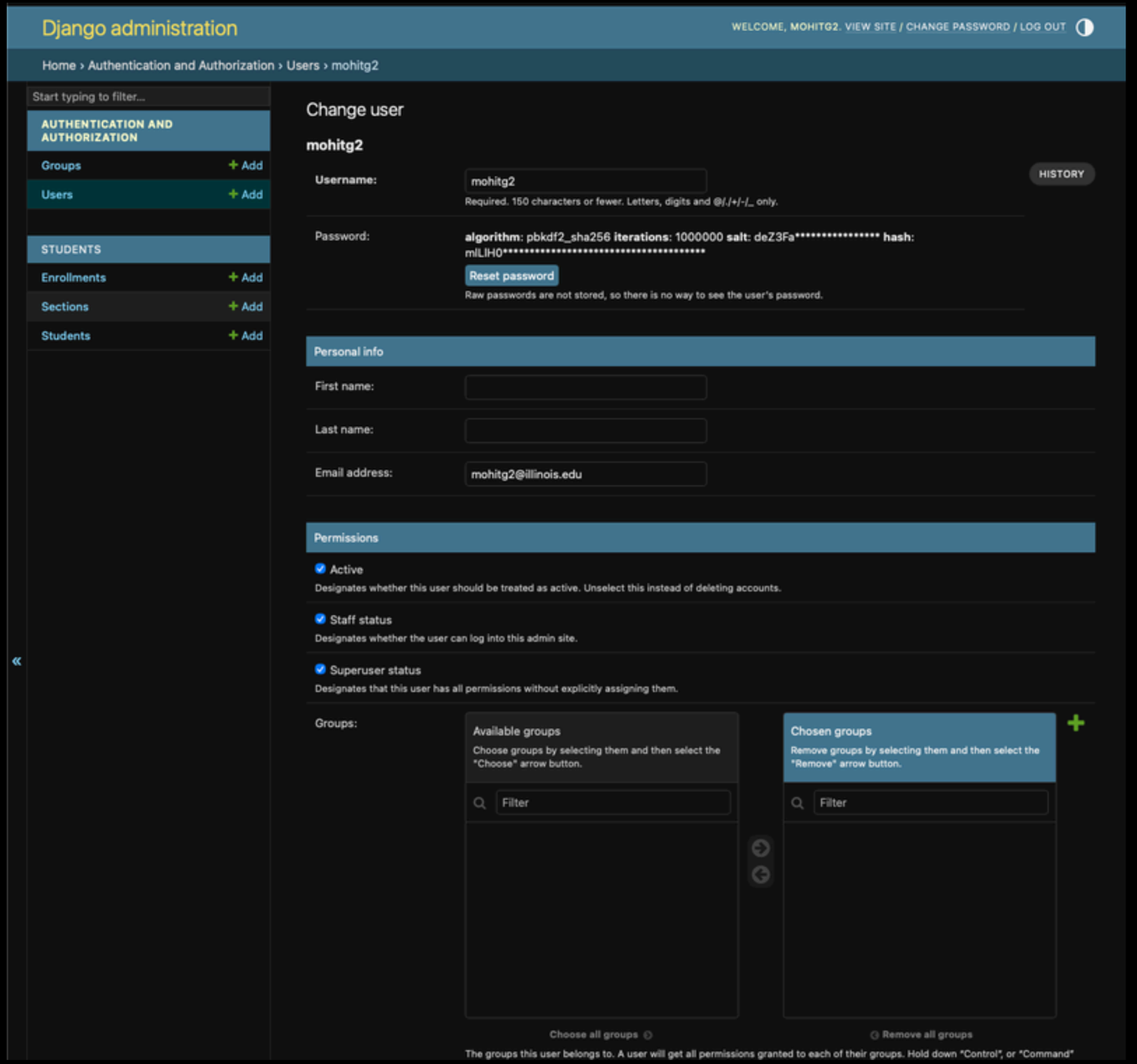
Registering the public users

(With no access to the admin page)

To learn the basics, we are going to store the users in the in-built “Users” model that Django gave us.



Now we are going to restrict access of public to this page.



List of fields the USER table has:

Field name	Description	Example
id	Auto Primary Key	1
password	Hashed Password	pbkdf2_.....
last_login	Datetime of Last Login	2025-10-26 20:22:00
is_superuser	Boolean	False
username	Chosen username	mohitg2
first_name	Optional	Mohit
last_name	Optional	Gupta
email	Email address entered via the form	mohitg2@illinois.edu
is_staff	For admin access	False
is_active	True by Default	True
date_joined	Timestamp	2025-10-26 20:22:00

```python
class StudentSignUpForm(UserCreationForm):
    """
    This form creates a new Django user (NOT a Student row).
    Fields:
    - username
    - password1
    - password2 (confirm password)
    We're also adding email to store contact info.
    """

    #
    email = forms.EmailField(
        required=True,
        help_text="We'll use this email if we need to contact you."
    )

    class Meta:

        # This is a pre-existing table in Django where we are already storing our internal user's data
        model = User
        fields = ["username", "email", "password1", "password2"]
```

- Creating a form to save signup data in the default User table.
- **We are also enabling mandatory requirement for the “email” field and a text for support message.**

```python
from .forms_auth import StudentSignUpForm

def signup_view(request):  2 usages (2 dynamic)
    """
    Show a sign-up form (GET),
    create an account (POST),
    then log the user in automatically.
    """
    if request.method == "POST":
        form = StudentSignUpForm(request.POST)
        if form.is_valid():
            # 1. Create the user in auth_user table
            new_user = form.save()

            # 2. Log them in immediately (no separate login step)
            login(request, new_user)

            # 3. Send them somewhere in the app
            return redirect("student-list-url")
    else:
        form = StudentSignUpForm()

    return render(request, template_name: "students/signup.html", context: {"form": form})
```

- We'll import StudentSignUpForm from the **forms_auth.py** file that we just created and validate the response.
- The intended file is **views.py**

- Lastly, setting the changes in the urls.py
- And adding **LOGIN_URL** variable in the **settings.py**
- Just by using **LOGIN_URL**, you can avoid multiple usages of **login_url = 'login_urlpattern' (like in the screenshot)**

```python
    # new addition/changes: auth routes
    path( route: "login/", LoginView.as_view(template_name="students/login.html"), name="login_urlpattern"),

    path( route: "logout/", LogoutView.as_view(next_page="login_urlpattern"), name="logout_urlpattern"),

    path( route: "signup/", views.signup_view, name="signup_urlpattern"),


]
```

```python
# new addition/changes: This automatically sets where to send people if they are not logged in
# settings.py
LOGIN_URL = "login_urlpattern"        # where to send people if not logged in

```

```python
# views.py
class StudentListView(LoginRequiredMixin, ListView):
    model = Student
    template_name = "students/student_list.html"
    login_url = 'login_urlpattern'      # required if LOGIN_URL not in settings
    redirect_field_name = 'next'        # optional (Django uses 'next' by default)
```

- Now creating a new signup form in **students/signup.html**

- **And adding the signup button in the existing students/login.html**

We are now just three steps away from a deployable project.

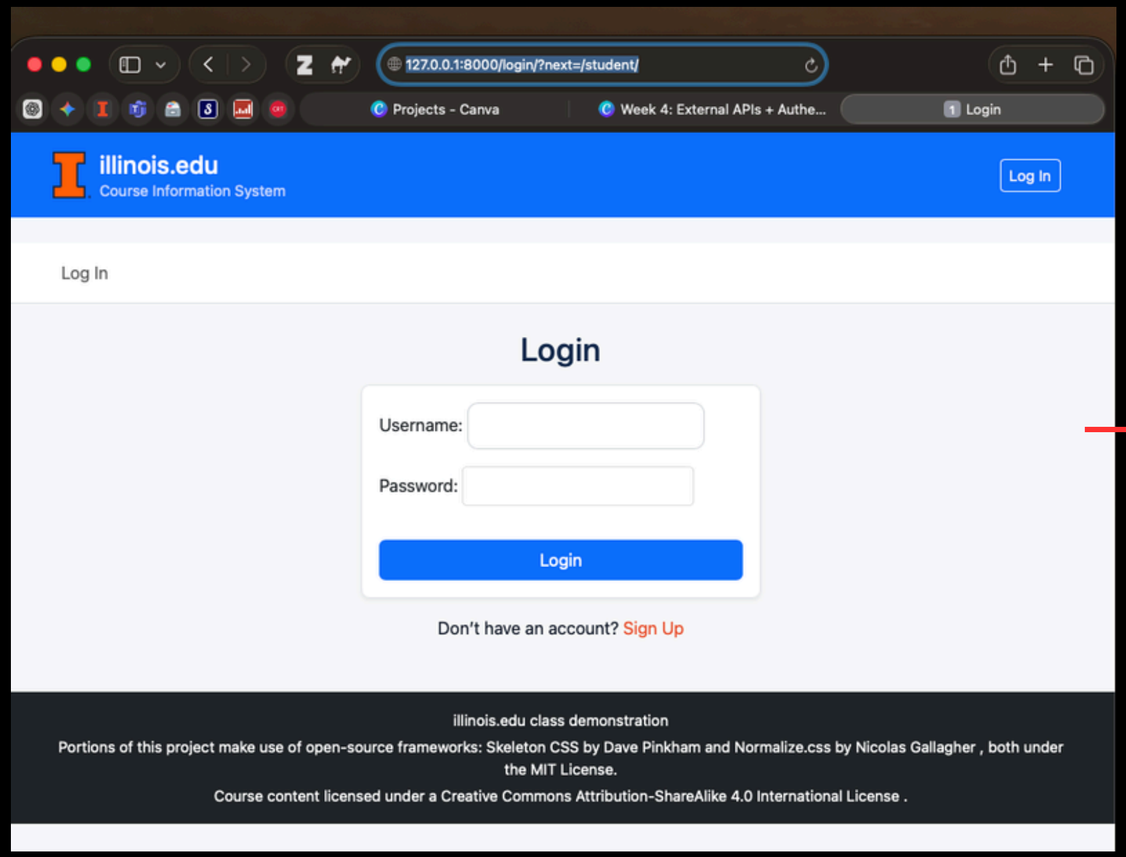
Activity time!

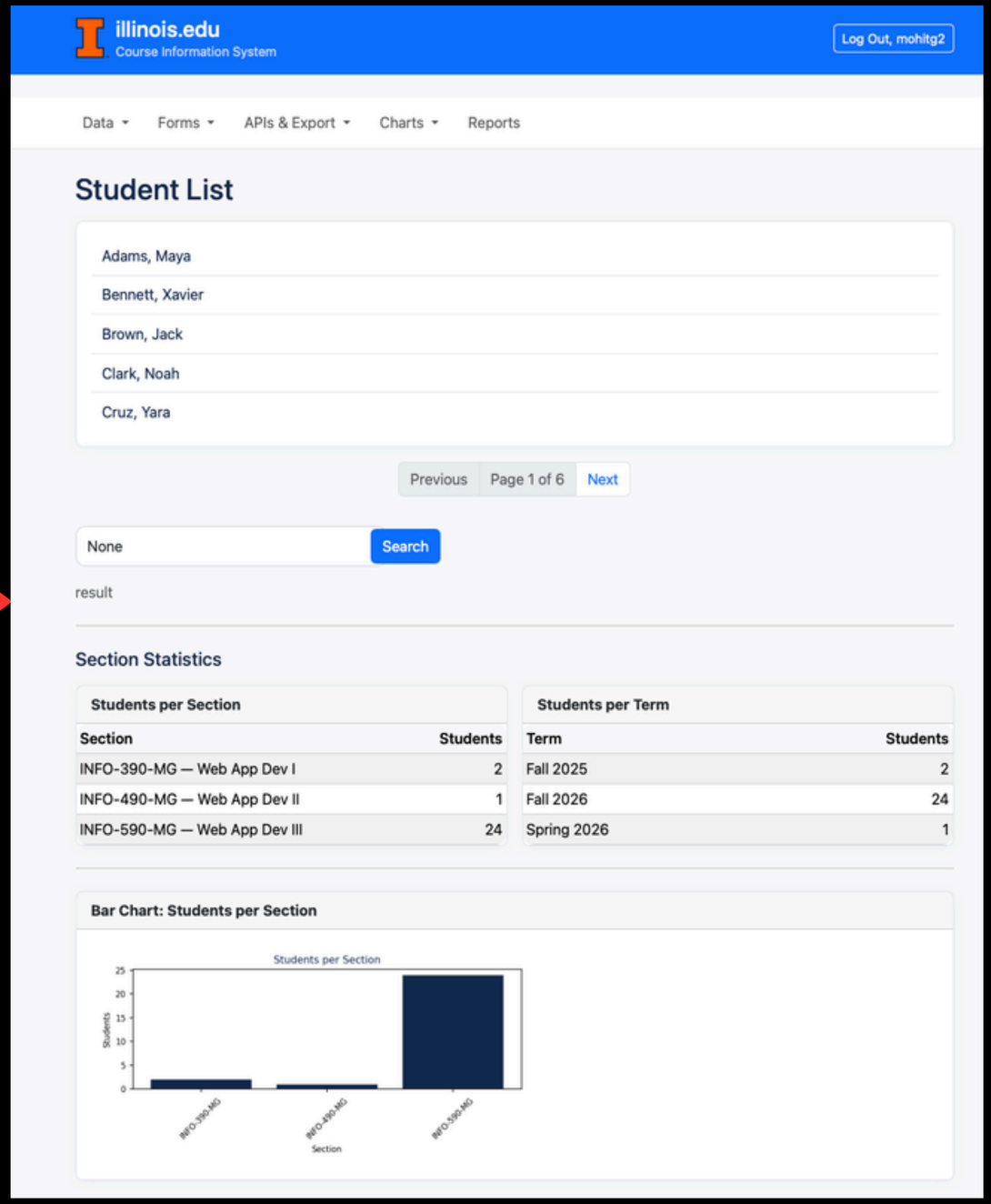
6. Code Clean-up

To prepare the website for deployment in the next class.

As we are nearing the end of Phase 1, let’s clean-up the code.

Now, login is mandatory.
Without it, you will not be able to get in.





THAT'S ALL FOR TODAY